

DelegationChain: Aggregatable Capability Chains for Cross-Organizational Agent Authorization

Daniyar Nurmukhamet*

School of Digital Technologies and Artificial Intelligence
D. Serikbayev East Kazakhstan Technical University
Öskemen, Kazakhstan

September 29, 2026

Abstract

Autonomous software agents built on large language models increasingly invoke tools and services across organizational boundaries, frequently via the Model Context Protocol (MCP). When an agent at one organization invokes a tool at another, the receiving organization currently has no cryptographically verifiable guarantee that the invocation was authorized, that the specific parameter values were sanctioned rather than just the action type, or that the authorization chain leading to the call was not tampered with. Existing authorization protocols address subsets of these properties but do not jointly satisfy them. We propose DelegationChain, a protocol providing parameter-bound, aggregatable authorization proofs that a receiving organization verifies locally, without a synchronous call to the issuing authority on the dominant path. The protocol uses BLS12-381 signatures with proof-of-possession, a recursive chain digest construction enabling compact aggregate signatures across N -hop delegations, a typed declarative policy expression language with a polynomial-time, sound scope containment check, and a federated identity registry model that permits interoperation between mutually distrusting organizations without a shared trust anchor. We define a threat model organized into six families of attack, describe the protocol’s cryptographic and operational components, and present a preliminary security analysis. We discuss limitations, including the absence at this stage of an empirical performance evaluation and machine-checked security proofs, and outline the path to a full evaluation including symbolic protocol verification and a Rust reference implementation.

1 Introduction

Software agents built on large language models are increasingly deployed in configurations where they invoke external tools and delegate sub-tasks to other agents. The Model Context Protocol (Model Context Protocol Project, 2026), introduced in late 2024, has emerged as a widely-adopted standard for this interaction pattern. As deployment scales, an architectural pattern is becoming common: agents operated by one organization invoke tools or delegate work to agents operated by another.

Cross-organizational agent invocation creates an authorization problem that existing infrastructure does not adequately address. Consider an agent at Organization A, acting on behalf of a human principal, that needs to invoke a payment-processing tool at Organization B. Organization B’s tool must decide whether to honor the request. To do so safely, B needs verifiable answers to four questions: (i) did the request actually originate from an authorized agent at A, rather

*Contact: `nurmuhamet.d@edu.ektu.kz`.

than from an attacker impersonating that agent; *(ii)* was the agent operating within the scope its human principals authorized, rather than acting beyond its delegated authority; *(iii)* were the specific parameter values supplied — the amount, the recipient, the file path — sanctioned by the authorization chain, or has an intermediate party substituted them; and *(iv)* has the authorization chain itself been preserved intact, or has some step in the chain been forged, dropped, or reordered.

Existing protocols address subsets of these questions but do not jointly satisfy them. OAuth 2.1 (Hardt et al., 2026) proves an agent’s identity to a resource server but does not bind authorization to specific parameter values, and where tokens are introspected rather than self-contained it requires a synchronous call to the authorization server on the verification path. Its answer to crossing organizational boundaries, the identity-chaining draft (Schwenkschuster et al., 2026), narrows scope at each boundary but costs a round trip to the authorization servers on both sides of every one. Macaroons (Birgisson et al., 2014) provide attenuated capability delegation with policy caveats and reach across trust domains through third-party caveats, at the cost of a round trip to each third party to collect a discharge. SPIFFE (SPIFFE Project, 2026) provides workload identity within a trust domain but does not include parameter binding or delegation chains. UCAN (UCAN Working Group, 2026) extends capability delegation to public-key settings and, in version 1.0, specifies a policy language over command arguments, but it defines no containment relation between capabilities and does not address MCP integration or BLS aggregation. Among MCP-specific work, AgentBound (Bühler et al., 2026) provides single-organization runtime enforcement, measurement work (H. Zhou et al., 2026) establishes the size of the gap, and Maloyan and Namiot (2026) propose a protocol extension that authenticates servers and their messages to clients; none provides a cross-organizational proof that an agent’s invocation was authorized. The Agent Identity Protocol (Prakash, 2026) is the closest design: it offers locally verifiable delegation chains built on Biscuit, but its tokens do not bind an invocation’s parameter values, and it has no registration or federation model and defers revocation. ACLE-MCP (Ding et al., 2026) issues a short-lived lease for each protected call that carries parameter bounds and attests the provider-side workload; its leases are issued per call rather than verified from a chain built once per delegation.

This paper proposes DelegationChain, an authorization protocol designed specifically for cross-organizational agent invocation. We describe a protocol design intended to combine, in a single coherent system, several properties that prior systems have addressed individually: public-key cryptographic verification across organizations without a shared root of trust; binding of authorization to exact parameter values, not merely action types; compact aggregate signatures across multi-hop delegation chains; and verification that requires no synchronous calls to the issuing authority during the dominant path. The protocol is constructed on BLS signatures (Boneh, Lynn, et al., 2001; Boneh, Bradley, et al., 2026) over the BLS12-381 curve (Sakemi et al., 2026), with proof-of-possession registration, a recursive chain digest construction enabling chain integrity under aggregation, a typed declarative policy language with termination and containment guarantees, and a federated organizational registry model.

Contributions. This paper makes four contributions. First, we define a threat model for cross-organizational agent invocation organized into six families of attack, spanning protocol-level attacks, identity infrastructure attacks, and threats specific to LLM-based agent systems such as prompt injection within permitted policy, and separating the families the protocol prevents from the one it merely bounds (Section 3). Second, we specify the protocol’s cryptographic core, wire format, verifier algorithm, and identity infrastructure (Sections 4 and 5) to a level of detail sufficient for a reference implementation, though not for byte-exact interoperability between independent ones, which would require the normative specification Section 4 declines to fix. Third, we introduce a typed declarative policy expression language with a formal grammar,

a polynomial-time scope containment procedure that is sound with respect to a semantic subset relation, and a proof of evaluator termination (Section 6), which we argue is sufficiently expressive for realistic enterprise authorization while remaining free of the injection and denial-of-service hazards of general-purpose policy languages. Fourth, we provide a preliminary security analysis (Section 7) reducing the protocol’s core security properties to standard cryptographic assumptions.

Scope and limitations. This paper is a design contribution. We do not yet present empirical performance measurements from a reference implementation, machine-checked security proofs, or results from symbolic protocol verification tools such as Tamarin or ProVerif. Each of these is in progress and is required before the protocol is ready for production use. We discuss the planned evaluation in Section 8.2. The contribution of this paper is the design and its preliminary analysis; the full evaluation is future work.

Paper organization. Section 2 reviews necessary background on BLS signatures and the MCP architecture. Section 3 defines the threat model. Section 4 describes the cryptographic core: chain digest construction, encoding, approval receipts, and the verifier algorithm. Section 5 describes the federated identity infrastructure. Section 6 introduces the policy expression language. Section 7 sketches the security argument. Section 8 discusses related work and limitations. Section 9 concludes.

2 Background

This section reviews the cryptographic and protocol background required to follow the design. Readers familiar with pairing-based cryptography and MCP may skim.

2.1 Model Context Protocol

The Model Context Protocol (MCP) (Model Context Protocol Project, 2026) is a JSON-RPC-based protocol standardizing how AI agents discover and invoke external tools. An MCP *host* (typically an LLM-based agent) communicates with one or more MCP *servers*, each exposing a set of tools. Tools are described by name, a typed parameter schema, and documentation. The host requests a tool invocation by sending a JSON-RPC message specifying the tool name and parameter values; the server executes the tool and returns the result.

MCP’s design assumes that the host and server are mutually trusted, an assumption appropriate for the single-organization deployments common in its early adoption. As MCP deployments scale to cross-organizational configurations, this assumption becomes untenable. A server invoked by an unknown agent has no cryptographic basis on which to decide whether the invocation is authorized. The protocol’s current security guidance relies on transport-layer authentication (typically OAuth) which addresses caller identity but not the broader authorization questions enumerated in Section 1. The gap is not hypothetical: a measurement study of real-world remote MCP servers (H. Zhou et al., 2026) identified 7,973 live servers and found that 40.55% expose tools without any authentication at all, and that among the OAuth-enabled servers it was able to probe, every one exhibited at least one authentication flaw.

That measurement predates the current revision of the specification, 2026-07-28 (Model Context Protocol Project, 2026). The revision deprecates Dynamic Client Registration, the source of the most prevalent flaw class Zhou et al. report, in favor of Client ID Metadata Documents. It also makes the core protocol stateless, removing the initialization handshake so that every request carries its own protocol version and client information. Neither change touches the question this paper addresses. The client information a request carries is self-declared, and

authorization remains a matter of OAuth tokens scoped to a server, not of evidence that a particular call was sanctioned.

2.2 BLS Signatures and Aggregation

The Boneh-Lynn-Shacham (BLS) signature scheme (Boneh, Lynn, et al., 2001) is a signature scheme constructed from bilinear pairings on elliptic curves. The scheme is specified for interoperable use by Boneh, Bradley, et al. (2026); the curve we adopt, BLS12-381, is specified separately by Sakemi et al. (2026). On that curve, signatures consist of a single point in $\mathbb{G}_2$ (96 bytes compressed) and public keys consist of a single point in $\mathbb{G}_1$ (48 bytes compressed). Signing requires one scalar multiplication; verification requires evaluation of a bilinear pairing.

The property that makes BLS signatures particularly attractive for our setting is *signature aggregation*. Given signatures $\sigma_1, \dots, \sigma_n$ on distinct messages $m_1, \dots, m_n$, their sum $\sigma_{\text{agg}} = \sum_i \sigma_i$ is itself a valid object in $\mathbb{G}_2$, and its validity with respect to the signing public keys and messages can be checked via a single multi-pairing operation. Aggregate signatures of this kind were introduced, and proved unforgeable for distinct messages, by Boneh, Gentry, et al. (2003). The CFRG draft’s basic scheme adopts the same rule, rejecting any aggregate in which two messages coincide (Boneh, Bradley, et al., 2026).

The precondition for aggregate unforgeability is that the *messages* be distinct; the signers need not be. The distinction matters for delegation chains, where the same agent may legitimately appear at more than one hop — a chain $A \rightarrow B \rightarrow A$ is a normal pattern, not an attack — and where the recursive chain digest of Section 4.3 guarantees distinct messages at every hop by construction, since each m_k commits to m_{k-1} . Aggregation has two consequences relevant to our design.

First, the bandwidth required to transmit n signatures is reduced to a single $\mathbb{G}_2$ element, regardless of n . For our delegation chains, this means a chain of arbitrary length carries a fixed 96-byte aggregate signature rather than $96n$ bytes of individual signatures.

Second, the computational cost of verification is reduced relative to verifying n signatures independently. We address the precise cost in Section 4.6; here we note that the cost is not, as is sometimes stated, asymptotically $O(1)$ in the number of signers when the signers sign distinct messages. The honest characterization, which we adopt throughout, is that aggregation provides significant constant-factor savings via Miller loop sharing in the underlying multi-pairing implementation.

The better-known hazard of BLS aggregation, the rogue-key attack, arises when signers sign a *common* message. An adversary who registers a key computed from other participants’ keys can then forge an aggregate on their behalf. Defenses for that setting include requiring proof-of-possession at registration (Ristenpart and Yilek, 2007) and the alternative analyzed by Boneh, Drijvers, et al. (2018). Neither is needed for aggregate security here, because distinct messages are themselves the rogue-key defense of the basic scheme (Boneh, Bradley, et al., 2026). We do require proof-of-possession at registration, but for a different reason (Section 5.3).

2.3 Capability-Based Authorization

The protocol design draws on the capability-based security tradition (Miller, 2006). In a capability system, authority is represented by unforgeable tokens (capabilities) that the holder can present to obtain access. Capabilities can be *attenuated*: a holder may produce a restricted version of a capability and delegate the restricted version to another party, without being able to grant authority exceeding what they themselves hold.

Macaroons (Birgisson et al., 2014) apply this model to web authorization. A macaroon is

an HMAC chain in which each link adds “caveats” — predicates restricting the macaroon’s use. First-party caveats are checked by the target service itself, and verification recomputes the HMAC chain using a secret that the target service knows.

Macaroons do address the cross-domain case, through *third-party caveats*. A caveat may name a principal other than the target service; to use the macaroon, the holder must obtain a *discharge macaroon* from that principal attesting that the caveat is satisfied, and present it alongside. The discharge is cryptographically bound to the macaroon it discharges, and the target service verifies it without sharing a secret with the third party. It is therefore not correct to say that Macaroons cannot cross organizational boundaries, and we do not say so.

The difference that matters for our setting is where the work happens. A third-party caveat costs a round trip: before presenting the macaroon, the holder must contact each named third party and collect a discharge, and the third party must be reachable at that moment. DelegationChain places the equivalent authority in signatures the verifier checks directly against registry-resolved public keys, with no discharge collection step on the request path.

UCAN (UCAN Working Group, 2026) extends capability attenuation to public-key settings, signing DAG-CBOR-encoded tokens in a Varsig-headed envelope with Ed25519, P-256 or secp256k1. UCAN is designed for Web3 contexts and lacks features specific to our setting: signature aggregation, a containment relation between capabilities (UCAN 1.0 does specify a policy language over command arguments), and MCP transport binding.

DelegationChain inherits the conceptual framework of attenuated capabilities from this tradition. Our contribution is the combination of public-key verification (extending Macaroons’ attenuation model across organizational boundaries), signature aggregation (improving over the per-hop linear growth of UCAN tokens), an explicit policy language with termination and containment guarantees, and a federated identity infrastructure.

3 Threat Model

This section defines the threat model under which DelegationChain operates. We state the protocol’s assets, the assumed adversary capabilities, the threats we defend against, the threats we explicitly do not defend against, and the security properties we seek to provide.

3.1 Assets and Principals

The assets we seek to protect are: (i) the integrity of tool invocations, in the sense that an accepted invocation faithfully represents the authorization granted by the originating principals; (ii) the provenance of authorization chains, in the sense that the chain delivered to a verifier accurately records the parties that authorized each step; and (iii) the integrity of audit records derived from accepted chains.

The principals participating in the protocol are: the *human operator*, who initiates a request and whose authority is the ultimate source of permissions; the *orchestrator agent*, which receives the initial authorization and may delegate to other agents; *specialist agents*, which receive delegated authority and either invoke tools directly or delegate further; the *tool* or *MCP server*, which receives authorization chains and decides whether to honor invocations; the *signing service*, a hosted component that holds an agent’s private key and applies policy enforcement before signing (it receives the body it is asked to sign, with the predecessor digest m_{k-1} , and computes m_k itself, since a service shown only a digest could enforce nothing); the *approval service*, which obtains human approval for actions requiring it and signs approval receipts; the *issuance service*, which signs session tokens for its organization; and the *registry*, which issues identity certificates binding each of these principals to a public key and a role.

3.2 Adversary Capabilities

We model a probabilistic polynomial-time adversary with the following capabilities. The adversary can observe and modify all network traffic between principals (a standard Dolev-Yao adversary on the network). The adversary can compromise an agent’s runtime environment, gaining the ability to instruct the signing service to sign messages of the adversary’s choice within the scope permitted by policy. The adversary can inject content into agent inputs to attempt prompt injection attacks. The adversary cannot, by default, compromise hardware security modules, the signing service infrastructure, the approval service infrastructure, the registry, or human approvers’ devices; threats involving these compromises are addressed separately and explicitly.

3.3 Threats Within Scope

The threats in scope fall into six families. Variants within a family share a *target* — what the attacker is trying to subvert — and not necessarily a defense; Section 7.3 maps each variant individually to the theorem or mechanism that answers it. Threats are labelled for cross-reference in the security analysis (Section 7).

We divide them by what the protocol actually does about them rather than by family. Section 3.3.1 lists what it *prevents*. Section 3.3.2 lists what it only *bounds*: the attack remains possible, and the protocol limits what it can achieve. Both are in scope, in the sense that the design is answerable for them, but a reader who took the second group for the first would credit the protocol with guarantees it does not provide. The division cuts across two families — T4a and T5b are bounded while the rest of T4 and T5 are prevented — and we would rather have an untidy grouping than a tidy overclaim.

3.3.1 Threats the protocol prevents

T1 — Forgery of tokens and chains. **T1a:** *Token forgery.* An adversary attempts to produce a valid authorization token without holding the corresponding signing key.

T1b: *Chain forgery and reordering.* An adversary attempts to construct an authorization chain that was not produced by the legitimate delegation sequence, including reordering hops, dropping hops, or inserting unauthorized hops.

T2 — Misrepresentation of granted authority. The two variants share a target — the authorization that reaches the verifier is not the authorization that was granted — but not a defense: T2a is defeated by the signature over the parameter encoding, T2b by the containment check.

T2a: *Parameter substitution.* An adversary, positioned between a signer and a verifier, attempts to alter parameter values in an authorized invocation (e.g., changing the destination of a transfer) while preserving the appearance of valid authorization.

T2b: *Scope escalation.* An agent attempts to delegate more authority than it was itself granted.

T3 — Reuse of valid tokens. **T3a:** *Replay across sessions.* An adversary captures a valid authorization chain and attempts to use it after its intended validity window has expired.

T3b: *Replay within validity window.* An adversary captures a valid authorization chain and attempts to use it more than once within its validity window.

T3c: *Cross-context token reuse.* An adversary attempts to reuse a token issued in one context (e.g., for a delegation) as if it had been issued in another context (e.g., for an invocation).

T3d: *Cross-verifier reuse.* An adversary presents a chain that one verifier accepted to a different verifier that accepts the same partner policy and exposes the same tool and action names. Examples are a production and a staging deployment, two regions with separate nonce caches, or two organizations that both pinned the same published policy.

T4 — Subversion of the approval path. The remaining variant of this family, T4a, is bounded rather than prevented and appears in Section 3.3.2.

T4b: *Approval routing attack.* An agent routes approval requests to a permissive approval service when policy requires a stricter one.

T4c: *Approval reuse across invocations.* An adversary attempts to reuse a valid approval receipt with an invocation different from the one approved.

T4d: *Stale approval after parameter modification.* An adversary attempts to modify invocation parameters after approval has been obtained.

T5 — Attacks on the identity infrastructure. The variant T5b is bounded rather than prevented and appears in Section 3.3.2.

T5a: *Key misattribution at registration.* An adversary attempts to have the registry certify, under the adversary’s own identifier, a public key it does not control — typically another principal’s. The adversary gains no ability to sign. What it breaks is the binding between keys and parties that non-repudiation depends on: the key’s real holder can disown signatures made under it.

T5c: *Revocation suppression.* An adversary suppresses revocation information from reaching verifiers, extending the window during which compromised credentials remain accepted.

T5d: *Certificate substitution.* An adversary substitutes a different certificate for the same identity during the registration or resolution process.

T5e: *Role confusion.* An adversary holding a key certified for one role — an agent, a signing service or an approval service — uses it to sign in another. For instance, it signs a session token naming itself as issuer, to obtain its organization’s full policy ceiling rather than the authority it was granted.

3.3.2 Threats the protocol bounds but does not prevent

The protocol limits the damage the following attacks can do, but does not prevent them, and we do not claim that it does. Section 7.3 states what the bound is in each case.

T4 — Subversion of the approval path (one variant). **T4a:** *Compromised approval service.* An adversary obtains the signing key of an approval service and issues fraudulent approvals. The protocol cannot prevent this: a valid signature under a registered approval key is, by construction, a valid receipt. What it does is contain the consequences. Approval receipts are verified separately from the chain aggregate (Section 4.5), and an approver’s certificate is accepted only for receipts, never for a session or delegation body (Section 4.6). A compromised approval service therefore cannot forge delegation chains, cannot enlarge any agent’s scope, and cannot attest to anything beyond what it was already trusted to attest to; the blast radius is exactly the set of invocations that policy routes through that service. Detection and recovery belong to the identity infrastructure — transparency log monitoring, revocation and rotation

(Section 5). Policies name approval services by identifier, and their keys are resolved and checked for revocation at verification time, so revoking a compromised approval key takes effect without a new policy version.

T5 — Attacks on the identity infrastructure (one variant). **T5b:** *Compromised registry signing key.* An adversary obtains a registry’s root signing key and issues fraudulent certificates. The protocol cannot prevent this, and the security arguments of Section 7 assume an honest root (Section 7.1). The damage is bounded in two ways. A registry root is trusted only for identifiers in its own organization’s namespace (Section 5.4), so the compromise cannot reach another organization’s identities. And sessions obtained through certificates it mints stay within the policies verifiers have pinned for that organization (Section 6). Recovery requires re-establishing the root out of band (Section 5.5).

T6 — Misuse of an agent within its own policy. **T6a:** *Prompt injection within policy.* An adversary, controlling content fed to an agent, induces the agent to take actions that are within the agent’s permitted policy but contrary to the principal’s intent.

T6b: *Confused deputy in delegation.* An agent is induced to sign a delegation token favoring an adversary by manipulation of the inputs the agent uses to construct delegation decisions.

3.4 Threats Outside Scope

The following threats are explicitly outside our scope. Defending against them requires mechanisms orthogonal to the protocol.

A fully compromised signing service can produce signatures on any messages within an agent’s policy; the protocol cannot prevent this and relies on operational hardening (HSM-based key custody, monitored access). Compromise of an approval service is treated in Section 3.3.2 rather than here, because the protocol does bound its effect even though it cannot prevent it. Side-channel attacks on cryptographic implementations are out of scope; we rely on the security properties of well-audited BLS libraries such as `blst`. Attacks on the human approver’s device or authentication mechanism are out of scope; the protocol assumes a trusted path between the approval service and the human. Denial-of-service attacks on network infrastructure are out of scope. Post-quantum security is out of scope for this version; BLS12-381 is not post-quantum secure, and migration to a post-quantum scheme is left to future work.

3.5 Security Properties Sought

We seek to provide the following security properties, each of which is formally stated and argued in Section 7:

Authenticity. Authorization tokens are unforgeable: no PPT adversary lacking access to a signing key can produce a token that verifies under the corresponding public key, except with negligible probability.

Parameter binding. A token authorizing an invocation with parameter values P is invalid for any parameter values $P' \neq P$.

Chain integrity. The aggregate signature over a delegation chain covers the entire chain content; modification, reordering, truncation, or extension of the chain invalidates the aggregate.

Scope monotonicity. In any valid chain, the session scope is contained in a policy the verifier has pinned for the issuing organization, and the scope at hop $k + 1$ is contained in the scope at hop k .

Non-repudiation. Neither party to a verified action can plausibly deny it, assuming honest signing key management.

Bounded revocation propagation. Once a registry has issued a revocation assertion, the additional window before verifiers cease accepting tokens under the affected certificate is bounded by a configurable parameter (default: 120 seconds). Detection of the compromise and its reporting to the registry precede this window and are properties of the deploying organization’s monitoring rather than of the protocol. The protocol bounds propagation, not total exposure.

4 Protocol Design

This section presents the cryptographic core of DelegationChain: the participants, the token types, the chain digest construction enabling aggregate signatures across delegations, the encoding of parameter-bound authorization, the handling of approval receipts, and the verifier algorithm. We describe the design at a level of detail sufficient to support the security analysis of Section 7 and to motivate the implementation effort. Bit-level details beyond those given here — field numbering within each structure, and the exact byte layout of the wire format — are not fixed by this paper and would be settled by a normative specification, which does not yet exist.

4.1 Participants and Tokens

A DelegationChain authorization chain has the following participants, in the order they contribute to the chain. The *issuer* I is the organization’s issuance service, holding a certificate of kind **issuer** from its registry, which issues an initial session token to the orchestrator agent at the start of an authorized session. The registry’s root key certifies the issuer but never signs sessions itself, so it can stay offline. The *orchestrator* A_1 is the agent that receives the session token and initiates the workflow. Intermediate *specialist agents* $A_2, \dots, A_{N-1}$ may be delegated sub-tasks. The *invoking agent* A_N is the agent that finally invokes a tool at the verifier. The *verifier* V is the receiving tool or MCP server. When policy requires human approval, an *approval service* contributes a separately-signed receipt.

The chain contains three token types. The *session token*, signed by the issuer, authorizes the orchestrator to operate within a scope for a bounded duration. *Delegation tokens*, signed by intermediate agents A_k for $1 \leq k < N$, attenuate authority and pass it to the next agent. The *invocation token*, signed by the final agent A_N , binds the invocation to specific tool, action, and parameter values. The chain delivered to the verifier consists of all token bodies plus a single aggregate signature in $\mathbb{G}_2$.

4.2 Cryptographic Primitives

We use BLS12-381 (Sakemi et al., 2026) with the minimal-pubkey-size variant of Boneh, Bradley, et al. (2026): public keys are points in $\mathbb{G}_1$ (48 bytes compressed), signatures are points in $\mathbb{G}_2$ (96 bytes compressed), and the pairing maps $\mathbb{G}_1 \times \mathbb{G}_2$ to $\mathbb{G}_T$. Hash-to-curve uses RFC 9380 with suite BLS12381G2_XMD:SHA-256_SSWU_RO_. SHA-256 is used for all non-curve hashing, including the chain digest construction.

Signing key $sk \in \mathbb{Z}_q$, public key $pk = sk \cdot g_1 \in \mathbb{G}_1$. For message $m \in \{0, 1\}^*$, $\text{Sign}(sk, m) = sk \cdot \text{HashToG}_2(m) \in \mathbb{G}_2$. Verification checks $e(g_1, \sigma) \stackrel{?}{=} e(pk, \text{HashToG}_2(m))$. For signatures $\sigma_1, \dots, \sigma_n$ on distinct messages — which the chain digest construction of Section 4.3 guarantees, and which does not require the signers to be distinct — the aggregate $\sigma_{\text{agg}} = \sum_i \sigma_i \in \mathbb{G}_2$ is verified by a multi-pairing check (Section 4.6).

Chain signatures use the basic scheme of the CFRG draft (Boneh, Bradley, et al., 2026), whose ciphersuite identifier is

BLS_SIG_BLS12381G2_XMD:SHA-256_SSWU_RO_NUL_

(the final NUL marks the basic scheme). Its aggregate verification rejects any aggregate in which two messages are equal, and its security for distinct messages is that of Boneh, Gentry, et al. (2003), which does not depend on how keys were registered. All participants also register their public keys with proof-of-possession (Section 5.3), for attribution rather than for aggregate security.

4.3 Chain Digest Construction

The central cryptographic construction is a recursive digest chain binding each hop to all prior hops. We use domain separation tags to ensure tokens of different types cannot be confused (preventing cross-context reuse, T3c). Tags are 8-byte ASCII strings with a null terminator: TAG_SES for sessions, TAG_DEL for delegations, TAG_INV for invocations.

Let $\text{Canon}(\cdot)$ denote canonical encoding (deterministic CBOR per RFC 8949 §4.2.1, with the additional rules summarized in Section 4.7).

Session message. The issuer signs:

$$m_0 = H(\text{TAG_SES} \parallel \text{Canon}(\text{SessionBody})) \quad (1)$$

where SessionBody contains the issuer’s identifier and public key (`issuer_id`, `issuer_pk`), the orchestrator’s identifier and public key (`subject_id`, `subject_pk`), a session identifier (`session_id`), a hash committing to the governing policy (`policy_hash`), a scope expression, issue and expiry timestamps, and a fresh nonce. The issue timestamp is informational. No check in the verifier depends on it, since, like every timestamp a signer supplies, it is not evidence of when signing happened (Section 5.6).

Delegation messages. For each intermediate hop A_k with $1 \leq k < N$:

$$m_k = H(\text{TAG_DEL} \parallel m_{k-1} \parallel \text{Canon}(\text{DelegationBody}_k)) \quad (2)$$

where DelegationBody_k contains the delegator’s identifier and public key (`delegator_id`, `delegator_pk`), the delegatee’s identifier and public key (`delegatee_id`, `delegatee_pk`), the attenuated sub-scope, its hop index k (`hop_index`), the parent session’s identifier (`session_id`), expiry, and a fresh nonce. The hop index and session identifier are redundant with the digest recursion, which already fixes each delegation’s position and session. The verifier checks them anyway, in phase 1, because doing so rejects a chain spliced from two sessions, or with a hop out of place, before any pairing is computed. The delegator’s identifier is carried explicitly so that a verifier can resolve the signer’s certificate from the body it is checking, rather than having to reach back into the preceding token. The critical property is that m_k commits to m_{k-1} via the hash, recursively committing to the entire chain history.

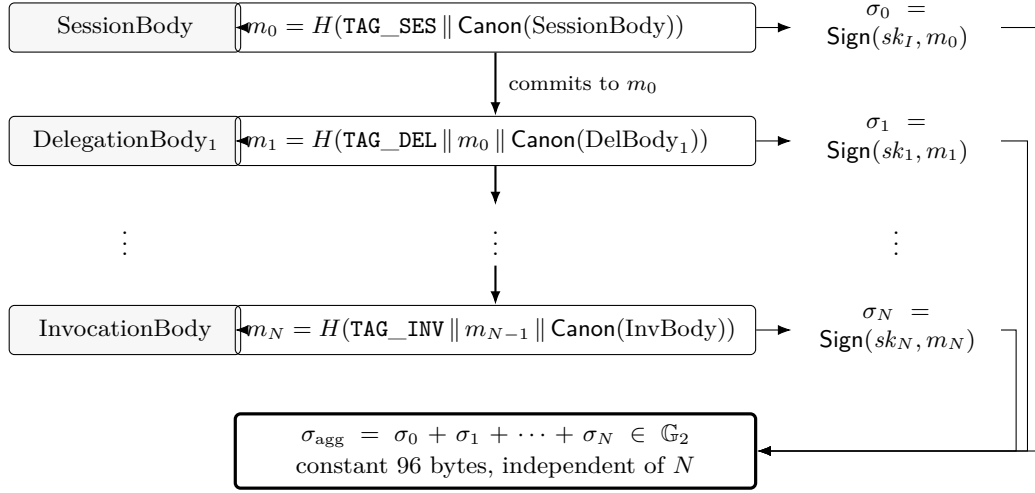


Figure 1: The recursive chain digest. Each hop’s message commits to its predecessor’s, so the order of hops is fixed by the digests themselves; the domain separation tag fixes each token’s kind. All hop signatures are summed into a single $\mathbb{G}_2$ element. The verifier recomputes $m_0, \dots, m_N$ from the received bodies, using the tag determined by each token’s position, and checks the sum in one multi-pairing.

Invocation message. The final hop A_N signs:

$$m_N = H(\text{TAG_INV} \parallel m_{N-1} \parallel \text{Canon}(\text{InvocationBody})) \quad (3)$$

where InvocationBody contains the invoker’s identifier and public key (`invoker_id`, `invoker_pk`), the identifier of the verifier the invocation is addressed to (`aud`, a `service` identifier; Section 5.2), the target tool and action identifiers, the canonically encoded parameter values, a hash of the parameters (defense-in-depth against encoding bugs), one approval receipt for each approval service the policy requires (possibly none; Section 4.5), and timing and nonce fields.

Figure 1 shows the construction as a whole.

Aggregate signature. Each participant signs its corresponding message with its private key, producing $\sigma_k = \text{Sign}(sk_k, m_k)$ for $k = 0, 1, \dots, N$. The aggregate signature transmitted to the verifier is

$$\sigma_{\text{agg}} = \sigma_0 + \sigma_1 + \dots + \sigma_N \in \mathbb{G}_2, \quad (4)$$

which has constant size 96 bytes regardless of N . The aggregate is accumulated along the chain: each participant receives the running aggregate from its predecessor, adds its own contribution, and forwards the sum. This does not keep the individual contributions secret — an adversary on the network (Section 3.2) observes successive partial sums and recovers any σ_k by subtraction — and nothing in the security argument requires that it should. Theorem 3 rests on the digest recursion, not on the secrecy of the summands.

Role distinguishability by position. The verifier reconstructs the m_k values from received token bodies using the tag determined by token position: `TAG_SES` for the session token, `TAG_DEL` for each intermediate delegation token, and `TAG_INV` for the invocation token. Every body also carries an explicit `kind` field, with value `session`, `delegation`, or `invocation`, inside its signed encoding. The verifier *rejects* chains where the position-derived tag does not match a body’s `kind` — preventing chain-extension attacks that would attempt to reuse an invocation token as a delegation, or vice versa. Such a body would fail the aggregate check in any case, since the verifier would hash it under a tag it was not signed with. The explicit field lets the verifier reject it in phase 1, before any pairing, and removes any ambiguity about how a body is to be parsed.

4.4 Parameter Binding

Parameter binding is achieved by including the canonical encoding of parameter values directly in the message m_N that the invoking agent signs. Any modification to parameter values — changing an amount, substituting a recipient, altering a file path — changes the canonical encoding, which changes m_N , which invalidates the signature contribution σ_N to the aggregate.

To support nested parameter structures (e.g., `order.payment.amount`), parameter values are encoded as a nested CBOR map with text-string keys sorted lexicographically by encoded form (per RFC 8949 §4.2.1). Parameter values are restricted to unsigned and signed integers, byte strings, NFC-normalized text strings, booleans, null, and nested maps and arrays of the same. Float values and tagged values are disallowed in parameter encodings, to eliminate edge cases around floating-point comparison and the introduction of unexpected types. Parameter map *keys* are text strings and are NFC-normalized on the same terms as text values. Without an explicit rule for keys, two implementations could canonically re-encode the same logical parameter map to different byte strings, and the re-encoding check of Section 4.7 would diverge between them.

Parameter binding is a cryptographic property, not an authorization one. It guarantees that the values a verifier reads are the values the invoking agent signed; it says nothing about whether those values were permitted. Whether they were is decided by the policy evaluation of Section 6.3, whose closed-world treatment of undeclared parameters is what turns binding into constraint.

The InvocationBody also includes `params_hash`, a SHA-256 digest of the canonical parameter encoding. The verifier recomputes this hash from the encoded parameters and rejects any chain where it does not match. This is redundant with the signature check but serves as defense-in-depth against subtle encoding bugs in implementations.

4.5 Approval Receipts

When policy requires human approval for an action, the InvocationBody carries an *approval receipt* from each approval service the policy requires, signed by that registered service. The receipt binds the human’s approval decision to the specific invocation being approved.

The flow is as follows. The agent constructs the InvocationBody *except* for the receipts field, computes the invocation digest of this preliminary body (defined below), and submits the digest along with the full body to the required approval service. The service presents the proposed invocation to a human approver. On approval, the service signs an *approval body* containing the service’s identifier and public key (`approver_id`, `approver_pk`), the invocation digest, opaque attestation data identifying the human approver, and issue and expiry timestamps (`iat`, `exp`). The signed receipt is returned to the agent, which embeds it in the now-complete InvocationBody and proceeds with chain construction.

Receipts are signed under their own domain separation tag, distinct from those of chain signatures and registration proofs. The kind check of phase 7 already stops a key from being used across these roles, so the separate tag is defense in depth.

The *invocation digest* is

$$\text{InvocationDigest}(B_N) = H(\text{TAG_IVD} \parallel \text{Canon}(B_N \text{ without its receipts field})).$$

The receipts field is omitted when there are no receipts, so this is exactly the digest of the preliminary body the agent submitted, and the tag separates it from m_N . Receipts are ordered by approver identifier, bitwise ascending, with at most one per approval service. A body that breaks either rule, or carries an empty receipts field, is malformed. A receipt from an approval service that the policy does not require for the invocation is ignored.

The approval signature is verified separately from the main aggregate. This decoupling has

two consequences. First, the security argument for chain integrity does not depend on approval service trust: compromise of an approval service does not enable chain forgery. Second, approval verification adds, per receipt, a hash to $\mathbb{G}_2$ and a two-pairing check (two Miller loops and one final exponentiation). Receipts are typically zero or one per invocation, and we judge this an acceptable cost.

Policy can require approvals from multiple services (logical AND); the required service identifiers are part of the policy that the session token’s `policy_hash` commits to. The policy names approval services by identifier only, never by key. Their keys are resolved at verification time like any other signer’s — `root`, `namespace`, `kind` `approver`, `expiry` and `revocation` (Section 5) — so revoking or rotating an approval service’s key takes effect without a new policy version and without every partner re-pinning. The policy hash is fixed at session start and does not vary with where the agent sends its approval request, so an agent that routes to a different (perhaps more permissive) service still presents the correct hash. What it cannot present is a receipt from a service the policy requires. The verifier, having resolved the policy that the hash commits to, and having checked that every scope in the chain is contained in it (Section 6), obtains an approval requirement no weaker than the policy’s own, and rejects a chain whose receipts do not cover it. This is the defense against the approval routing attack (T4b).

4.6 Verification

The verifier algorithm proceeds in phases, ordered to fail fast on cheap checks before performing expensive cryptographic operations. Algorithms 1 and 2 state the procedure; the paragraphs that follow comment on individual phases. Every **reject** is terminal. The only decision-relevant state the procedure changes is the nonce cache, in its final step, and that step is an atomic insert-if-absent. Resolution and policy loading may fill caches (Section 5.4). That affects the cost of later verifications but not, within the propagation bound of Section 5.6, their outcome. A chain that fails any check therefore leaves the nonce cache exactly as it found it.

Three abbreviations keep the statement short, all of them reading fields named in Section 4.3. Write $\text{sid}(B_k)$ for the identifier of the party that signed B_k : `issuer_id` for the session body, `delegator_id` for a delegation body, `invoker_id` for the invocation body. Write $\text{spk}(B_k)$ for that party’s public key as named in the body: `issuer_pk`, `delegator_pk` or `invoker_pk`. Every body names its signer’s key, so during a scheduled rotation, when two certificates for one identifier are valid at once, the key determines which one applies. Write $\text{scope}(B_k)$ for the body’s scope expression: the granted scope for the session body, the attenuated sub-scope for a delegation body. Finally, $\text{org}(x)$ is the `organization-id` prefix of an identifier x (Section 5.2); $\text{Root}[o]$ is the registry root key the verifier holds for organization o (Section 5.1); and $\text{Pinned}[o]$ is the set of policy hashes the verifier has accepted from organization o (Section 6). The certificate kind that position k requires is $\text{role}(k)$: `issuer` for $k = 0$, `agent` for $k \geq 1$. *self* is the verifier’s own `service` identifier.

Structural and temporal checks. The verifier first decodes the received chain and confirms canonical encoding (rejecting non-canonical inputs to prevent encoding-ambiguity attacks), that the recomputed `params_hash` matches (Section 4.4), that every delegation body carries its own position as `hop_index` and the session’s `session_id`, and that the invocation is addressed to this verifier: the invocation body’s `aud` must equal the verifier’s own `service` identifier. It then checks that the current time is within the invocation’s validity window, that expiry times are monotonically non-increasing along the chain (each hop’s expiry cannot exceed its parent’s), and that the chain has minimum length one.

Algorithm 1 DelegationChain verification, phases 1–5: structural, temporal, and identity checks. Phases run cheapest-first; every **reject** is terminal. Continued in Algorithm 2.

```

1: procedure VERIFY(received chain  $C$ , current time  $t$ )
  ▷ Phase 1 — structure and encoding
2:    $(B_0, \dots, B_N, \sigma_{\text{agg}}) \leftarrow \text{DECODE}(C)$ ; reject if decoding fails
3:   reject if  $N < 1$ 
4:   for all  $B_k$  do
5:     reject if  $\text{Canon}(B_k) \neq$  received bytes of  $B_k$            ▷ non-canonical encoding
6:   end for
7:   reject if any  $B_k.\text{kind}$  disagrees with the tag its position implies
8:   reject if  $B_N.\text{aud} \neq \text{self}$                                    ▷ addressed to this verifier
9:   reject if  $H(\text{Canon}(B_N.\text{params})) \neq B_N.\text{params\_hash}$ 
10:  for  $k \leftarrow 1$  to  $N - 1$  do
11:    reject unless  $B_k.\text{hop\_index} = k$  and  $B_k.\text{session\_id} = B_0.\text{session\_id}$ 
12:  end for
  ▷ Phase 2 — temporal
13:  reject if  $t \notin [B_N.\text{nbv}, B_N.\text{exp}]$ 
14:  for  $k \leftarrow 1$  to  $N$  do
15:    reject if  $B_k.\text{exp} > B_{k-1}.\text{exp}$            ▷ expiry must not grow along the chain
16:  end for
  ▷ Phase 3 — replay
17:  reject if  $(\text{spk}(B_N), B_N.\text{nonce}) \in \text{NonceCache}$ 
  ▷ Phase 4 — key chain consistency
18:  reject if  $(B_0.\text{subject\_id}, B_0.\text{subject\_pk}) \neq (\text{id}(B_1), \text{spk}(B_1))$ 
19:  for  $k \leftarrow 1$  to  $N - 1$  do
20:    reject if  $(B_k.\text{delegatee\_id}, B_k.\text{delegatee\_pk}) \neq (\text{id}(B_{k+1}), \text{spk}(B_{k+1}))$ 
21:  end for
  ▷ Phase 5 — identity resolution
22:  for  $k \leftarrow 0$  to  $N$  do
23:     $\text{cert}_k \leftarrow \text{RESOLVE}(\text{id}(B_k), \text{spk}(B_k))$ ; reject if unresolvable
24:    reject unless  $\text{cert}_k$  verifies under  $\text{Root}[\text{org}(\text{id}(B_k))]$ 
25:    reject unless  $\text{cert}_k.\text{registry\_id} = \text{org}(\text{id}(B_k))$            ▷ registry owns the namespace
26:    reject unless  $\text{cert}_k.\text{kind} = \text{role}(k)$                        ▷ role fixed by position
27:    reject if  $\text{cert}_k$  is not yet valid, expired, or revoked at  $t$ 
28:     $\text{pk}_k \leftarrow \text{cert}_k.\text{pk}$                                    ▷ equals  $\text{spk}(B_k)$  by resolution
29:  end for

```

Replay protection. The verifier maintains a bounded-time nonce cache indexed by (invoker public key, nonce). Chains whose key is already present in the cache are rejected as replays. A chain is accepted only if inserting its key into the cache succeeds, with TTL equal to the chain’s remaining validity window plus a clock-skew tolerance. Line 13 applies no tolerance of its own: a chain is valid exactly within $[B_N.\text{nbv}, B_N.\text{exp}]$ as the verifier’s clock reads it, and senders must allow for skew when they set those bounds. The tolerance in the TTL has a different purpose. Instances of one logical verifier that share the cache may disagree about the time. The added term must be at least that disagreement, so that no instance evicts a chain’s entry while another could still accept the chain.

The lookup in phase 3 is only an early filter. What actually prevents replay is that the final insertion is atomic and fails if the key is already present. The lookup and the insertion are separated by identity resolution, which may involve a network call, and by the pairing computation. Without atomicity, two copies of one chain presented concurrently would both pass

Algorithm 2 DelegationChain verification, phases 6–8: policy, approvals, and the aggregate check. Continues Algorithm 1; line numbering and the enclosing procedure carry over.

```

    ▷ Phase 6 — policy
30:   reject if  $B_0.\text{policy\_hash} \notin \text{Pinned}[\text{org}(\text{sid}(B_0))]$                                 ▷ verifier's own choice
31:    $S \leftarrow \text{LOADPOLICY}(B_0.\text{policy\_hash})$ ; reject if unavailable
32:   reject if  $\text{CONTAINS}(S, \text{scope}(B_0))$  is false                                          ▷ the ceiling
33:   for  $k \leftarrow 1$  to  $N - 1$  do
34:     reject if  $\text{CONTAINS}(\text{scope}(B_{k-1}), \text{scope}(B_k))$  is false
35:   end for
36:    $d \leftarrow \text{EVALUATE}(\text{scope}(B_{N-1}), B_N.\text{invocation})$ 
37:   reject if  $d = \text{DENY}$ 
    ▷ Phase 7 — approvals
38:   if  $d = \text{ALLOW-WITH-APPROVAL}(svcs)$  then
39:     for all  $s \in svcs$  do
40:        $R \leftarrow$  the receipt in  $B_N$  with  $R.\text{approver\_id} = s$ ; reject if none
41:        $\text{cert}_R \leftarrow \text{RESOLVE}(s, R.\text{approver\_pk})$ ; reject if unresolvable
42:       reject unless  $\text{cert}_R$  passes the phase-5 checks with kind approver
43:       reject unless  $R$  verifies under  $\text{cert}_R.\text{pk}$  over  $\text{INVOCATIONDIGEST}(B_N)$ 
44:       reject if  $t \notin [R.\text{iat}, R.\text{exp}]$ 
45:     end for
46:   end if
    ▷ Phase 8 — aggregate signature
47:   recompute  $m_0, \dots, m_N$  from  $B_0, \dots, B_N$  using position-determined tags
48:   reject if  $m_i = m_j$  for some  $i \neq j$                                                 ▷ basic-scheme precondition
49:   reject unless  $e(g_1, \sigma_{\text{agg}}) = \prod_{k=0}^N e(pk_k, \text{HashToG}_2(m_k))$ 
    ▷ Commit
50:   reject unless  $\text{INSERTIFABSENT}(\text{NonceCache}, (pk_N, B_N.\text{nonce}))$                     ▷ atomic
51:   accept
52: end procedure

```

phase 3 and both be accepted, on a single instance as readily as across replicas. The nonce cache is kept per verifier. Replay across *different* verifiers is prevented not by the cache but by the audience field: a chain names its verifier in **aud**, and any other verifier rejects it in phase 1. The cache size is bounded by the product of maximum verification throughput and the maximum validity window.

The cache must be a single logical structure. A deployment that places several verifier replicas behind a load balancer, each holding its own independent cache, does not provide replay protection at all: the same chain presented to two replicas is accepted twice, and an adversary who can influence load-balancer routing can replay at will. Replicated verifiers must therefore share a cache or partition it deterministically by invoker public key. This is a coordination dependency on the verification path, and Theorem 5 is stated with it as an explicit hypothesis.

Public key chain consistency. The verifier checks that each body is signed by the party its predecessor named. The session body's subject identifier and key must equal the signer identifier and key of B_1 , and each delegation body's delegatee identifier and key must equal those of the body that follows it. Comparing identifiers as well as keys ensures that the party that acts is the one that was named, even where one key is certified under more than one identifier. The signer of a delegation body is its delegator, and that of the invocation body is its invoker, so the rule also covers the shortest chain: when $N = 1$ there are no delegations, and the orchestrator named in the session is itself the invoker.

Scope monotonicity and policy compliance. The verifier first confirms that the session token’s `policy_hash` is one it has pinned for the issuing organization (Section 6). It then obtains the policy document, from its cache or by hash from the issuing registry (Section 5.4), and establishes the containment chain Eq. (6) using the procedure of Section 6.4: the session scope must lie within the policy, and each hop’s sub-scope within its parent’s. Finally it evaluates the invocation against the innermost scope, rejecting if the invocation falls outside it. The pinning check is what gives the first link its force. The issuer signs both the policy hash and the session scope, so without pinning the verifier would be checking the issuer against a policy the issuer had selected.

Approval verification. If policy requires approvals, the verifier finds, for each required approval service, the receipt that names it. It resolves the service’s certificate through the same checks as phase 5, requiring kind `approver`. It then confirms that the receipt verifies under the certified key over the correct invocation digest, and that the current time lies within the receipt’s own validity window. A chain missing any of these is rejected.

Aggregate signature verification. Finally, the verifier reconstructs $m_0, m_1, \dots, m_N$ from the received bodies (using position-determined tags). It confirms that they are pairwise distinct, as the basic scheme’s aggregate verification requires. The digest recursion makes a repeat impossible short of a SHA-256 collision, so for a well-formed chain the check never fails; it is there so that the precondition of Theorem 3 is something the verifier establishes rather than assumes. It then checks the aggregate via a multi-pairing:

$$e(g_1, \sigma_{\text{agg}}) \stackrel{?}{=} \prod_{k=0}^N e(pk_k, \text{HashToG}_2(m_k)). \quad (5)$$

If this check succeeds, all integrity, parameter-binding, and chain provenance properties hold simultaneously.

Cost. The aggregate check is a multi-pairing with $N + 2$ pairings: one against g_1 and one for each of the $N + 1$ hops. Each hop also needs a hash to $\mathbb{G}_2$, to compute $\text{HashToG}_2(m_k)$. In modern BLS libraries (e.g., `blst`) the Miller loops of a multi-pairing share part of their computation, and a single final exponentiation is required. The cost is therefore approximated by $\alpha + \beta N$. Here α collects the final exponentiation and the terms that do not grow with N , and β is the per-hop cost: one hash to $\mathbb{G}_2$ plus one Miller loop. Hashing to $\mathbb{G}_2$ is not cheap — it involves a map to the curve and cofactor clearing in $\mathbb{G}_2$ (Faz-Hernández et al., 2023) — so β cannot be assumed small relative to α . For chains of more than a few hops, the linear term need not be dominated by the constant one, and which dominates in practice is for the measurements of Section 8.3 to settle. This compares favorably with verification approaches requiring network round-trips per hop (e.g., OAuth token introspection), but does not amount to truly asymptotic $O(1)$ verification; the honest characterization is “compact aggregate signatures with multi-pairing efficiency.”

One comparison should be stated before a reader finds it. The CFRG draft itself reports that verifying a single BLS signature is markedly slower than verifying an ECDSA signature at the same security level (Boneh, Bradley, et al., 2026). Its figures come from older hardware and implementations than current libraries such as `blst`, but the direction of the gap is not in doubt. It is the reason the case for aggregation has to be settled by measurement against a non-aggregating baseline (Section 8.3) rather than by the operation counts above.

Figure 2 shows why the order matters. The phases are arranged so that the checks a hostile chain is most likely to fail are also the cheapest to run: a malformed or expired chain is rejected on parsing and integer comparison, and only a chain that has survived every structural, temporal,

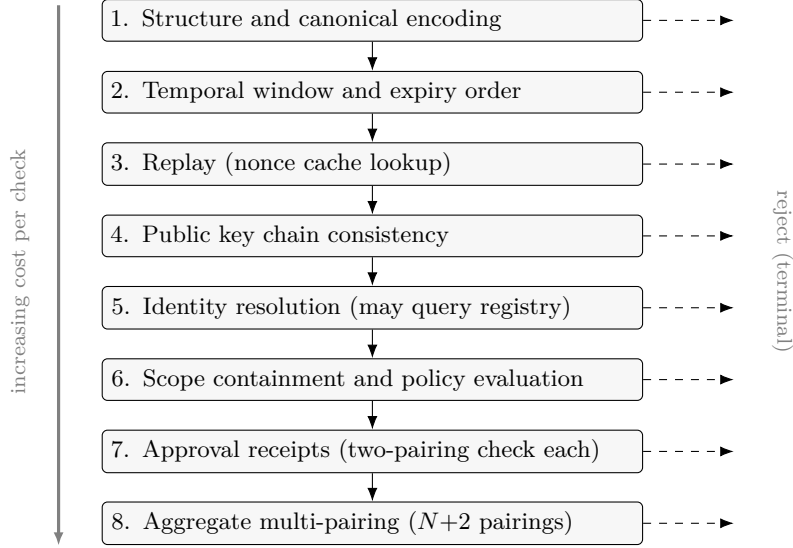


Figure 2: Phase ordering in Algorithms 1 and 2. Cheap checks run first, so that a chain which cannot succeed is discarded before the verifier performs any pairing. No state that affects a later decision changes until the final phase has passed; certificate and policy caches may be filled earlier.

and policy check reaches the pairing computation. An adversary who wishes to make a verifier do the expensive work must first construct a chain that is well-formed, unexpired, non-replayed, key-consistent, scope-monotone, and policy-compliant — at which point the aggregate check is the only thing left to establish, and the work is warranted.

4.7 Encoding

All signed structures use deterministic CBOR encoding per RFC 8949 §4.2.1, with the following additional rules. In *protocol structures* — session, delegation and invocation bodies, certificates, and approval receipts — map keys are integers rather than text strings, eliminating UTF-8 normalization considerations and reducing encoding size. Parameter maps are the exception: they carry names chosen by the tool author rather than by this specification, and so use text-string keys sorted lexicographically by encoded form, per Section 4.4. The scoping is deliberate, and an implementation that applies the integer-key rule to parameter maps will not interoperate with one that does not. Float and tagged values are disallowed in protocol structures except where explicitly required. Group elements are encoded using the standard ZCash-format compressed serialization defined for BLS12-381, and decoders perform subgroup membership checks on every group element to defend against small-subgroup attacks. The pairing-friendly-curves draft asks protocols adopting its serialization to state three further choices (Sakemi et al., 2026). Only the compressed form is accepted, so each point has exactly one encoding. The identity element is rejected wherever a public key or signature is decoded: for public keys this is the key-validation rule of the signature draft (Boneh, Bradley, et al., 2026), and an honest chain produces an identity aggregate only with negligible probability. No scalar appears in any protocol structure, since secret keys never leave their holders; a zero secret key is excluded at registration, because its public key is the identity.

Verifiers reject any structure whose received encoding differs from the canonical re-encoding of the decoded content. This precludes a class of attacks based on adversarially-crafted non-canonical encodings that parse correctly but hash to different values than the implementation expects.

5 Identity Infrastructure

The cryptographic protocol of Section 4 assumes that verifiers can resolve identity-to-public-key bindings for all participants in a chain. This section describes the federated identity infrastructure providing those bindings, including registration with proof-of-possession, public-key resolution, key rotation, and revocation.

5.1 Federated Registry Model

DelegationChain does not assume a global root of trust. Each participating organization operates its own *registry*, which issues identity certificates for agents, signing services, and approval services within that organization. Each registry holds a long-lived root keypair, with the root public key serving as the trust anchor for all certificates the registry issues.

Two organizations establish trust through one of three mechanisms, chosen per deployment. *Direct peering* exchanges registry root keys out-of-band as part of a business agreement; this is the simplest model and adequate for small numbers of partners. *Federated transparency* publishes registry root keys and certificate issuances to a shared append-only log, modeled on Certificate Transparency (Laurie et al., 2021) but adapted to DelegationChain’s certificate format; verifiers trust the log’s commitments and obtain inclusion proofs binding claimed keys to log entries. *Hierarchical attestation* introduces an attestation authority signing statements binding registry roots to organizational identifiers, analogous to a root CA. The protocol permits all three; we recommend direct peering or federated transparency over hierarchical attestation, the latter inheriting the trust-concentration concerns of Web PKI.

These mechanisms are not mutually exclusive. A production deployment may use direct peering with frequent partners and a federated log for the long tail. Whatever the mechanism, the peering relationship is also where the receiving organization fixes which of its partner’s policies it will honor. Its verifiers pin those policy hashes and reject sessions naming any other (Section 6).

5.2 Identifiers and Registration

Identifiers are structured as `organization-id:kind:local-id`, where `kind` is one of `agent`, `signer`, `approver`, `issuer` (the issuance service that signs session tokens), or `service` (a tool or MCP server acting as a verifier). The `kind` tag, which certificates also carry, prevents confusion across roles. The verifier enforces it rather than leaving it as a convention: each position in a chain, and each approval receipt, must be signed under a certificate of the matching kind (Section 4.6). A `service` identifier names a verifier and serves only as the audience of an invocation (Section 4.3). A verifier never has to prove its identity in this protocol, so it needs no certificate. The `signer` kind identifies a signing service in its own right, for operational authentication outside this protocol, for example between an agent runtime and the signing service that holds its key. A signing service signs chain bodies only with the agent keys it holds, under those agents’ certificates; no position in a chain, and no receipt, accepts a `signer` certificate. The `organization-id` matches the issuing registry’s identifier.

Registration produces a certificate binding an identifier to a public key, signed by the registry. Certificates contain version information, the identifier, the public key, the kind, the registry’s identifier (`registry_id`) and public key, issuance and expiry timestamps, a not-before timestamp, a serial number, and the registry’s signature. Default certificate lifetimes are short — 24 hours to 7 days for agents and signing services, with longer lifetimes permitted only in combination with active revocation (Section 5.6). Short lifetimes reduce reliance on revocation by ensuring that compromised credentials naturally expire within a bounded window.

5.3 Proof of Possession

Proof-of-possession (PoP) is often presented as a defense against rogue-key attacks on BLS aggregation, and in the setting where such attacks work — many signers on a common message — it is one (Ristenpart and Yilek, 2007). That is not our setting. An adversary who registered a key $pk^* = pk' - \sum_i pk_i$, built from chosen targets pk_i and a key pk' it controls, could forge an aggregate on a message the targets had all signed. No such message exists in a chain. Every aggregate in DelegationChain is over pairwise distinct messages, which is itself the rogue-key defense of the basic scheme (Boneh, Gentry, et al., 2003; Boneh, Bradley, et al., 2026), so chain integrity (Theorem 3) does not depend on PoP.

We require PoP at registration for a different reason: attribution. A PoP shows that the registrant controls the secret key for the public key it asks the registry to certify. Without it, an adversary could register another principal’s public key under its own identifier (T5a). The adversary would gain no ability to sign, but the registry would then certify one key to two parties. A signature verifying under that key would no longer identify who made it, and that identification is what non-repudiation (Section 3.5) rests on.

DelegationChain requires every registration to include a PoP signature over a registry-issued challenge. The challenge includes the registrant’s claimed identifier and public key, the kind tag, the registry’s identifier, a registry-issued fresh nonce (single-use, short expiry), and a timestamp. The registrant computes a BLS signature over this challenge with the secret key corresponding to the public key being registered. The registry verifies the PoP signature before issuing the certificate. The registry-issued nonce prevents replay of PoPs captured from other contexts. The PoP signature is computed under a domain separation tag distinct from the one used for chain signatures (Section 4.2), so a registration proof can never be presented as a chain signature, or the reverse. Ristenpart and Yilek (2007) show that proofs of possession as simple as a signature over the certificate request suffice against rogue-key attacks on BLS-based multisignatures. Our PoP is of that simple kind, though as argued above we rely on it for attribution rather than aggregate security.

Registries additionally perform subgroup membership checks on registered public keys, rejecting points that do not lie in the prime-order subgroup of $\mathbb{G}_1$. This check is non-trivial on BLS12-381 and has been the source of real-world implementation vulnerabilities. Implementations should use library-provided subgroup checks (e.g., `blst_p1_in_g1`) rather than reimplementing them.

5.4 Public Key Resolution

Verifiers resolve an identifier, together with the public key that the body being checked names for it, by querying the issuing registry’s resolution endpoint. Resolution returns the most recently issued certificate binding that identifier to that key, whether or not it is currently valid. Whether the certificate is valid at verification time — not yet valid, expired, or revoked — is decided by the verifier, at line 27 of Algorithm 1, not by the resolver. During the overlap of a scheduled rotation (Section 5.5) two certificates for one identifier are valid, and the key selects between them; this applies to issuers, agents and approval services alike. The response includes the certificate and, if the registry is integrated with a transparency log, an inclusion proof. The verifier validates the certificate signature under the root public key it holds for the organization the identifier names, that is, for the identifier’s `organization-id` prefix, bound to a root key by whichever trust mechanism of Section 5.1 the two organizations use. It also requires the certificate’s `registry_id` to equal that `organization-id`, and the certificate’s kind to be the role the identifier is used in: `issuer` for a session token, `agent` for a delegation or invocation, `approver` for a receipt. If an inclusion proof is present, the verifier checks it against the log’s current signed tree head.

This binding between organization identifier and root key carries real weight. A registry is authoritative only for identifiers in its own organization’s namespace. The naming rule of Section 5.2 is enforced here, by the verifier, rather than left as a convention. Without it, a registry the verifier trusts for one organization could issue a certificate for another organization’s identifier: a certificate substitution (T5d). And because the verifier looks up pinned policies by the organization named in the session body (Section 6), whoever controlled that registry could impersonate the other organization’s issuer and grant sessions under the policies pinned for it.

Verifiers cache resolved certificates to avoid registry queries on every verification. Cache entries are valid until the earlier of the certificate’s expiry and a configurable cache TTL (typically one hour). Cache invalidation is event-driven for revocation: observation of a revocation assertion immediately evicts the corresponding cached entry.

Policy resolution. The verifier must also hold the policy that a session token’s `policy_hash` commits to; Section 4.6 checks the session scope against it using the containment relation of Section 6.4, Section 6.3 then evaluates the invocation against the innermost scope, and Theorem 6 depends on that policy being one the verifier accepted, not one the chain selected. Since the policy is written by the sending organization and enforced by the receiving one, it has to get across the boundary somehow, and the protocol should not leave this unsaid.

Registries host policy documents alongside certificates, retrievable by hash: a verifier holding `policy_hash` h requests the document h names, and accepts it only if $H(\text{Canon}(\text{document})) = h$. Because resolution is content-addressed, the registry is not trusted for integrity — it cannot substitute a different policy, only decline to serve one — which is the same posture the rest of Section 5 takes towards registries. Policies are immutable under their hash, so a document once fetched is cached for as long as the verifier keeps it; a policy change produces a new hash and, hence, a new session token rather than a silent change under an existing one.

Pinning (Section 6) keeps this fetch off the request path. Because a verifier accepts only hashes it has pinned, it knows every hash it will need before any chain names one, and it can fetch and check the documents when it pins them. A hash it has not pinned is rejected without any fetch. A verifier that defers fetching to first use pays one round trip on that call; Section 8.2 discusses what the remaining dependencies cost the offline-verification claim.

5.5 Key Rotation

The protocol supports three rotation modes. *Scheduled rotation* issues a new certificate with overlapping validity windows: during the overlap, both old and new keys are valid, ensuring that chains in flight at the moment of rotation continue to verify. Default rotation cadence is every 30 days for agents and every 90 days for signing services and approval services. Because policies name approval services by identifier rather than by key (Section 4.5), rotating an approval service’s key needs no policy change. *Emergency rotation* truncates the validity of the prior certificate via an immediately-issued revocation assertion (Section 5.6) and issues a new certificate without overlap. *Registry root rotation* is the most sensitive operation: the current root signs a rotation attestation binding the new root to the old, which is then distributed via the same channel used for the original root. Compromised-root recovery requires out-of-band re-bootstrap of trust and cannot be handled by in-band rotation.

Rotation interacts with chains that are in flight when a key changes. Under scheduled rotation the overlapping validity windows cover this case: a chain signed under the previous key continues to verify for the duration of the overlap, because the previous certificate is still valid. Under emergency rotation there is no overlap, and chains signed under the withdrawn key are rejected. The verifier always resolves the certificate valid at *verification* time. It does not attempt to

resolve the certificate that was valid when a token was signed, because its only evidence of signing time would be a field in the token body, signed by the very key whose compromise is at issue; Section 5.6 develops this point.

5.6 Revocation

DelegationChain prefers short-lived certificates over revocation infrastructure: with default certificate lifetimes of 24 hours to 7 days, the maximum window during which a compromised key can be misused is bounded by certificate expiry. Explicit revocation is provided as a defense-in-depth mechanism for deployments choosing longer-lived certificates.

A *revocation assertion* is a signed statement by a registry declaring that a specific certificate is revoked, identified by serial number. Verifiers learn of assertions through polling (default interval 60 seconds), push notification, or transparency log monitoring; the choice is a deployment decision. The end-to-end window between key compromise and unforgeability is bounded by the sum of detection latency, reporting latency, assertion issuance, propagation, and residual cache time, of which only the propagation and cache terms are bounded by configuration (typically below 120 seconds). Detection and reporting latency are unbounded by the protocol: they depend on the deploying organization’s monitoring, and nothing in the design constrains them. The property claimed in Section 3.5 is accordingly bounded revocation *propagation*, not bounded exposure.

Revocation applies relative to *verification time*. A certificate revoked at time t_r is not accepted by any verifier that has learned of the revocation, including for chains whose tokens were signed before t_r . The gentler alternative — continuing to honor chains signed before t_r , so that revocation does not retroactively invalidate work already in progress — is not available to us. A verifier’s only evidence of signing time is a timestamp in the token body, and that timestamp is signed by the key whose compromise prompted the revocation. An adversary holding a stolen key, observing the revocation, can set the field to any earlier value and produce a chain that verifies against the pre-revocation certificate. A revocation rule resting on that field can be defeated by backdating, which would leave the mechanism without force.

We therefore accept the operational cost. Chains in flight when a revocation is issued may fail verification and must be reconstructed under a rotated key. Two things bound the exposure: only chains outstanding within the propagation window are affected, and scheduled rotation (Section 5.5) avoids the situation entirely by overlapping the validity windows of the old and new certificates, so that routine key changes never invalidate work in flight. Emergency rotation, by construction, does.

6 Policy Expression Language

This section introduces the policy expression language used to describe agent authorization scopes throughout DelegationChain. The language serves two roles, and separating them is worth doing at the outset, because the verifier treats them differently.

A *policy* is a scope expression published by an organization for its agents and committed to by the `policy_hash` field of the session token (Section 4.3). It is a ceiling: the outer bound on what any agent operating under that session may do, fixed by the organizations involved rather than by any agent. The session names which policy applies; the verifier decides which policies it will accept at all, as described below, and resolves the document from the registry by hash (Section 5.4). A *scope* is a scope expression carried inside a token — the authority the issuer granted the orchestrator, in the session token, and the attenuated sub-scope each delegator passed on, in a delegation token.

Both are drawn from the same grammar, and a single relation connects them. A chain is accepted only if

$$S \supseteq \text{scope}(B_0) \supseteq \text{scope}(B_1) \supseteq \cdots \supseteq \text{scope}(B_{N-1}), \quad (6)$$

where S is the resolved policy and $B_0, \dots, B_{N-1}$ are the session and delegation bodies. Every link is checked by the containment procedure of Section 6.4; the verifier then evaluates the invocation against $\text{scope}(B_{N-1})$, the innermost scope.

The first link needs care, because the issuer signs both `policy_hash` and $\text{scope}(B_0)$ in the same body. If the verifier accepted whatever hash a session named, the check would compare the issuer against a policy the issuer had chosen. Content addressing prevents a different document from being substituted behind a hash, but it does nothing about which hash is selected. The verifier therefore accepts only policies it has *pinned* for the issuing organization: a set of policy hashes fixed in its own configuration, or in the peering agreement with that partner (Section 5.1). A session naming any other hash is rejected. The ceiling S is then drawn from a set the receiving organization agreed to, and the issuer can do no more than choose among policies the verifier has already accepted.

Pinning also closes a gap the protocol would otherwise leave: the receiving organization had no way to state its own terms, only to enforce the sending organization's. A pinned policy may be one the sending organization published and the receiving organization reviewed, or one the two wrote together for the relationship. Either way, it is never one the receiving organization has not seen. What pinning does not defend against is the verifier's own configuration. We treat the pinned set as part of the verifier's trust base (Section 7.1).

With pinning in place, every link of Eq. (6) belongs to the scope monotonicity property of Section 3.5, and Theorem 4 states it for the whole chain.

One consequence carries the weight of Theorem 6. Definition 1 requires a contained scope's approval requirement to be at least as restrictive as its container's, so approval requirements can only strengthen as one moves right along Eq. (6). An approval requirement written into the policy therefore cannot be weakened by the issuer, nor by any agent further down the chain. This is what lets the verifier evaluate against $\text{scope}(B_{N-1})$ — data the agents wrote — and still be certain the requirement it obtains is no weaker than the one in a policy it has itself accepted.

The design of the policy language is constrained by three requirements arising from its role in the protocol. First, the verifier must be able to check cheaply, given two scope expressions S_1 and S_2 purportedly in a delegation relationship, whether $S_2 \subseteq S_1$ — the scope monotonicity check of Section 4.6 — erring towards rejection where it cannot establish containment. Second, evaluation of an invocation against a scope must terminate in time bounded by the size of the scope expression and the invocation, with no possibility of unbounded computation. Third, scope expressions must be free of side effects: evaluation must not perform network calls, file I/O, or other operations that could be exploited for information exfiltration or denial of service.

These requirements rule out general-purpose policy languages such as Rego or Cedar, which are powerful but operationally expensive to reason about and historically have included features (e.g., HTTP fetches in some versions of Rego) that violate the side-effect-free requirement. DelegationChain's policy language is deliberately restricted to a fragment that supports realistic enterprise authorization conditions while admitting straightforward decidability and termination analyses.

6.1 Grammar

The grammar of scope expressions, in Backus-Naur form:

```
scope ::= rule+ | "allow" "all" | "deny" "all"
rule  ::= "allow" at tool action params? where? approval?
```

```

at      ::= "at" "=" principal
tool    ::= "tool" "=" identifier
action  ::= "action" "=" identifier
params  ::= "params" "{" binding ("," binding)* "}"
binding ::= path ":" type
where   ::= "where" expr
expr    ::= expr "and" expr
          | "(" expr ")"
          | atom
atom     ::= path numop value
          | path strop string-value
          | path "in" list-value
          | path "under" string-value
approval ::= "approval" "requires" principal ("," principal)*
type     ::= "int" | "string" | "bool"
numop    ::= "<" | "<=" | "==" | ">=" | ">"
strop    ::= "==" | "starts_with" | "ends_with" | "contains"
path     ::= identifier ( "." identifier)*
principal ::= identifier ":" identifier ":" identifier
identifier ::= letter (letter | digit | "_" | "-")*
value     ::= integer | string | boolean
list-value ::= "[" value ("," value)* "]"
string-value ::= string

```

A scope is either a special form (**allow all**, **deny all**) or a non-empty disjunction of *rules*. Each rule specifies the verifier it authorizes (**at**), a tool, an action, an optional declaration of expected parameter types (**params**), an optional condition on parameter values (**where**), and an optional approval requirement (**approval**). A *principal* is a registry identifier of the form **organization-id:kind:local-id** (Section 5.2); its kind must be **service** in an **at** clause and **approver** in an **approval** clause, and a policy that breaks either rule is malformed. The semantics of a scope is the union of the invocations permitted by its rules; **deny all** permits nothing, and **allow all** is a reserved form used for top-level trusted contexts (e.g., the issuer's session-issuance authority). It permits every audience as well as every tool and action, so the mandatory audience clause does not constrain it. A verifier that pins **allow all** for a partner accepts that partner's sessions for anything addressed to it, with no policy limiting them, so it is not meant to be pinned across organizations.

The condition language inside **where** clauses is *conjunctive*: a clause is a conjunction of atomic predicates over parameter values. Each atomic predicate is one of four forms — numeric comparison, string operator, list membership (**in**), or path-prefix containment (**under**). No predicate involves function calls, network operations, or unbounded iteration.

The three parameter types are disjoint at the value level. Each corresponds to a different CBOR major type in the parameter encoding of Section 4.4 — integers, text strings, booleans — so every parameter value has at most one type, and a leaf value of any other encoded type (a byte string, null or an array) matches no declaration. The disjointness matters. If two types shared values, two rules declaring the same parameter under different types could both match one invocation, and the containment procedure of Section 6.4 would be unsound; a type for filesystem paths, whose values are text strings, would do exactly that. Path-prefix containment is therefore an operator on strings rather than a type. **p under q** holds when the value of string parameter **p** is a canonical absolute path and the segments of **q** are a prefix of its segments, equality included. A canonical absolute path starts with **/**; it is either **/** alone or **/** followed by non-empty segments joined by **/**; no segment is **.** or **..**; and there is no trailing **/**. A non-canonical value makes the atom false. The canonical-form condition is not optional: without it, **/home/agent/workspace/../../etc/passwd** would pass a check for **under "/home/agent/workspace"**. The literal **q** must itself be a canonical absolute path. A policy containing a non-canonical one is malformed and rejected when it is loaded, since such a prefix

would otherwise silently match nothing.

Lexical details. Whitespace is insignificant. Strings are double-quoted, with JSON escapes. Integers are decimal, with an optional leading `-`, and lie in the CBOR integer range $[-2^{64}, 2^{64} - 1]$. Booleans are `true` and `false`. In identifiers, `letter` is `[A-Za-z]` and `digit` is `[0-9]`, which rules out Unicode confusables and makes normalization irrelevant for identifiers. String literals are NFC-normalized, like the text values they are compared with (Section 4.4), and the string operators compare byte-wise.

Well-formedness. The grammar admits scopes that make no sense, and the procedures of Sections 6.3 and 6.4 are defined only for well-formed ones. A rule is *well-formed* when three conditions hold.

1. Every path its `where` clause mentions is declared in its `params`. No path is declared twice, and no declared path is a strict prefix of another (`a` together with `a.b`).
2. Every atom pairs its operator, the declared type of its path and its operand as in the table below. Any other combination is malformed.
3. The principal in its `at` clause has kind `service`, and those in its `approval` clause have kind `approver`.

Operator	Path type	Operand
<code>< <= >= ></code>	<code>int</code>	integer
<code>==</code>	<code>any</code>	a literal of the path’s type
<code>starts_with ends_with contains</code>	<code>string</code>	string
<code>in</code>	<code>any</code>	a non-empty list, every element of the path’s type
<code>under</code>	<code>string</code>	a canonical absolute path

The operator `==` appears in both `numop` and `strop`; the declared type of the path decides which is meant. A scope is well-formed when all of its rules are, and a scope that violates any of these conditions is *malformed*. `LOADPOLICY` rejects a malformed policy. A malformed scope inside a session or delegation body makes that body malformed, and the chain is rejected when it is decoded, at line 2 of Algorithm 1. The containment procedure of Section 6.4 also returns false on a malformed scope, as a defensive check that a decoded chain never reaches. Implementations also bound the size of a scope — its rules, parameters, atoms, list lengths, text lengths and path depth — so that evaluation and containment stay bounded on input the sender chooses; those bounds belong in a normative specification. One consequence is used in Proposition 2. In a well-formed rule, every path the clause mentions is declared, so it is present in any invocation that passes the rule’s closed-world check, and step 1(c) of the evaluator cannot fail.

Disjunction and negation are deliberately absent from the expression language. Disjunction is recovered one level up: a scope is a union of rules, so a condition $a \vee b$ is written as two rules, and the expressive loss is close to nil. Negation is recovered only in part. Numeric comparisons have complements within the language — the negation of $x > c$ is $x \leq c$ — but the string, membership, and path-prefix predicates do not, so a condition such as “the path is *not* under */etc*” cannot be written. We accept this. Authorization policies are more naturally written as enumerations of what is permitted than as exclusions of what is not, and the restriction is what buys the containment procedure of Section 6.4 its polynomial running time: implication between arbitrary propositional formulas is coNP-complete, and a verifier is not a good place to run a SAT solver on an input the sender chose.

6.2 Example

A representative policy fragment governing a payments tool:

```
allow at=orgb:service:payments
  tool=payments action=transfer
  params { amount: int,
           to_account: string }
  where amount <= 1000
allow at=orgb:service:payments
  tool=payments action=transfer
  params { amount: int,
           to_account: string }
  where amount <= 100000
    and to_account in
      ["acct_vendor_a", "acct_vendor_b"]
  approval requires orga:approver:finance
allow at=orgb:service:files
  tool=files action=read
  params { file: string }
  where file under "/home/agent/workspace"
```

This policy permits, at two named services of Organization B, unconditional transfers up to 1,000 units; transfers up to 100,000 units to two approved vendors with finance approval; and read access to files anywhere under a specific workspace directory. Any invocation falling outside these three rules is denied by the implicit closing rule **deny all**.

6.3 Evaluation Semantics

Given a scope S and an invocation $I = (\text{aud}, \text{tool}, \text{action}, \text{params})$, whose audience is the **aud** field of the invocation body, the evaluator returns **ALLOW**, **ALLOW-WITH-APPROVAL**(*services*), or **DENY**. The special forms are decided directly: **allow all** returns **ALLOW** and **deny all** returns **DENY**. Otherwise S is a list of rules, and the procedure is:

1. For each rule r in S in declaration order:
 - (a) If r 's audience, tool, or action does not match I , continue.
 - (b) Type-check $I.\text{params}$ against r 's **params** declaration. The check is *closed-world*: the rule matches only if $\text{dom}(I.\text{params})$ is exactly the set of parameter paths r declares, and every value has its declared type. If a declared parameter is missing, if any parameter has the wrong type, or if the invocation carries a parameter r does not declare, continue.
 - (c) Resolve each path mentioned in r 's **where** clause against $I.\text{params}$. If any path resolves to absent, continue (treating the rule as inapplicable).
 - (d) Evaluate r 's **where** clause. If false, continue.
 - (e) If r has no approval requirement, return **ALLOW**. Otherwise return **ALLOW-WITH-APPROVAL**(r 's required approval services).
2. If no rule matched, return **DENY**.

The closed-world condition. Step 1(b) is closed-world by design: an invocation carrying a parameter that the rule does not declare matches no rule, and is denied. The alternative — checking declared parameters and ignoring the rest — would leave undeclared parameters cryptographically bound but never authorized. Under Section 4.4 every supplied value sits inside

$\text{Canon}(P)$ and is therefore signed and non-repudiable, but binding is not constraint. A rule declaring `amount` and `to_account` says nothing about a third parameter `override_limits`, and a verifier that accepted an invocation carrying it would be honoring an authorization the policy never granted. Closing the world is what makes a parameter declaration a constraint rather than a schema hint.

The audience clause. Every rule names the verifier it authorizes, and step 1(a) requires that name to equal the invocation’s `aud`. The clause is mandatory for the same reason the parameter check is closed-world. The `aud` field of Section 4.3 binds an invocation to one verifier, but binding is not constraint. Without the clause, `tool=payments` would authorize payments at every organization that pinned the policy. A prompt-injected agent could then address a fresh, correctly signed chain to the wrong one and remain within policy. A policy used with several partners therefore lists its rules per verifier.

Match order. Rules are tried in declaration order and the first match wins, so a broad early rule silently suppresses the approval requirement of a narrower rule later in the scope. In the policy of Section 6.2 a 500-unit transfer to `acct_vendor_a` matches rule 1 and returns `ALLOW` without ever reaching rule 2’s finance approval. This is a policy-authoring hazard rather than a protocol defect — the outcome is exactly what the written policy says — but it is easy to introduce by accident, and authors should order rules from most restrictive to least, or scope early rules narrowly enough that they cannot shadow a later approval requirement.

The evaluator is deterministic, has no side effects, and contains no loops over data of unbounded size: the outer loop iterates over rules (a finite syntactic component of S), and `where` clause evaluation is a finite recursive descent over the expression tree.

Proposition 1 (Termination). *Evaluation of an invocation against a scope S terminates in time $O(|S| \cdot |\text{params}|)$, where $|S|$ is the syntactic size of the scope expression and $|\text{params}|$ is the total size of the invocation’s parameter encoding.*

Proof sketch. The outer loop iterates over rules of S , of which there are at most $|S|$. For each rule, parameter resolution traverses the parameter map, which is bounded by $|\text{params}|$; `where` expression evaluation is a recursive descent whose depth is bounded by the syntactic depth of the expression, itself bounded by $|S|$. The total work is bounded by the product, yielding the claimed bound. $\square$

6.4 Subset Relation

The verifier of Section 4.6 must check that the sub-scope at each delegation hop is a subset of its parent’s scope. This is a check over scope expressions, performed once per hop at verification time, and not a runtime check over invocations. We define the relation semantically and then give a syntactic procedure that soundly under-approximates it.

Definition 1 (Subset relation). Given scope expressions S_1, S_2 , we say $S_2 \subseteq S_1$ iff every invocation I that the evaluator accepts under S_2 is also accepted under S_1 , with at least as restrictive an approval requirement.

Containment is checked — conservatively, as Proposition 2 and Remark 1 make precise — by the following procedure, which first returns false if either scope is malformed (Section 6.1):

1. If S_1 is `allow all`, return true.

2. If S_2 is **deny all**, return true. If S_2 is **allow all** (and, by step 1, S_1 is not), return false.
3. For each rule r_2 in S_2 :
 - (a) Find the first rule r_1 of S_1 , in declaration order, such that $r_2 \subseteq r_1$ as rules. If there is none, return false.
 - (b) Consider every rule r' that precedes r_1 in S_1 , agrees with r_2 on audience, tool, action, and parameter declaration, and has a **where** clause jointly satisfiable with r_2 's. Skip r' if some rule r'_2 preceding r_2 in S_2 agrees with r' on audience, tool, action and declaration and has a clause implied by r' 's. Otherwise check that r' 's approval requirement is no stronger than r_2 's, and return false if it is stronger.
4. Otherwise, return true.

Step 3(b) is needed because rules are matched in declaration order (Section 6.3). An invocation accepted under S_2 is evaluated under S_1 against the *first* rule of S_1 that matches it. Since r_1 matches every invocation r_2 does, that first rule is r_1 or one of the rules before it. Rules after r_1 never decide such an invocation, and step 3(b) does not examine them. Without the check, a child scope could quietly drop an approval requirement imposed by an earlier parent rule — exactly the escalation Definition 1 is meant to exclude — and the procedure would not notice.

The skip condition handles the converse case, in which a rule of S_2 is itself shadowed. An invocation that r_2 decides under S_2 matched no earlier rule of S_2 , and in particular did not match r'_2 . Suppose it matched r' . It would then pass r'_2 's closed-world check, since the declarations are identical; every path in r'_2 's clause would be present, since in a well-formed rule those paths are declared; and it would satisfy that clause, which r' 's implies. So it would match r'_2 , which it did not. Hence r' cannot be the rule that decides it under S_1 . Without the skip, the procedure would reject policies written in the order Section 6.3 recommends. With an approval-requiring rule placed before an overlapping permissive one, such a policy would fail to contain itself. Phase 6 would then reject every session whose scope is the pinned policy, and every agent that re-delegates its whole scope unchanged.

With both conditions the procedure is reflexive: $P \subseteq P$ for every scope P . For each rule r_2 of P , step 3(a) succeeds, since r_2 subsumes itself, and the r_1 it finds lies at or before r_2 . Every rule r' preceding r_1 is therefore an earlier rule of P itself, and it is skipped with $r'_2 = r'$. The incompleteness of Remark 1, which concerns rules covered only by unions of other rules, remains.

Rule subsumption ($r_2 \subseteq r_1$) requires: the audiences, tools and actions match; r_1 and r_2 declare the *same* set of parameter paths with the same types; r_1 's approval requirement is no stronger than r_2 's (so that any approvals satisfying r_2 also satisfy r_1); and r_2 's **where** clause implies r_1 's.

The identity condition on parameter declarations follows from the closed-world rule of Section 6.3 together with the disjointness of types (Section 6.1). A rule matches only invocations whose parameter set is exactly what it declares and whose values have exactly the declared types, and each value has at most one type. The declaration of any rule matching an invocation I is therefore determined by I alone, and two rules with different declarations — different parameter sets, or the same parameters under different types — accept disjoint sets of invocations: declaring fewer parameters is strictly more restrictive rather than more permissive, and no containment holds in either direction. Requiring identical declarations is therefore the only sound relation available without further machinery, such as an explicit notion of optional parameters that the language does not currently have.

The remaining sub-problem is deciding implication between **where** clauses. Because a clause is a conjunction of atoms (Section 6.1), this reduces to a check over atoms rather than to satisfiability. The atoms have a restricted structure: numeric comparisons are over a fixed integer path and constants, string operators are over fixed string paths and constant patterns,

list membership is over fixed lists, and path-prefix containment is over fixed prefixes. Atoms over distinct paths are independent, so a conjunction $C_2 = \bigwedge_j a_j$ implies a conjunction $C_1 = \bigwedge_i b_i$ exactly when, for each b_i , those atoms of C_2 over the same path as b_i jointly imply b_i . For each of the four atom forms this is a closed-form check on the constraint the atoms induce on that path: an interval for numeric comparisons, a set of admissible values for membership, and a prefix or substring relation for the string and path operators. Joint satisfiability, needed by step 3(b), is the same analysis: a conjunction is unsatisfiable exactly when the constraints induced on some path are inconsistent. No search over truth assignments is required at any point.

Proposition 2 (Soundness and cost of the containment procedure). *The procedure above terminates in time polynomial in $|S_1| + |S_2|$, and whenever it returns true, $S_2 \subseteq S_1$ in the sense of Definition 1.*

Proof sketch. Cost. The procedure enumerates pairs $(r_2, r_1) \in S_2 \times S_1$, of which there are at most $|S_1| \cdot |S_2|$. Each pair check compares two identifiers, compares two parameter declarations for equality, compares two approval requirements, and decides implication between two conjunctions of atoms. The last of these groups the atoms of each clause by path and, for each atom of r_1 's clause, evaluates a closed-form implication against the constraint that r_2 's atoms induce on the same path; the number of such checks is bounded by the product of the two atom counts, and each is constant-time in the atom's operands. Step 3(b) performs a satisfiability check of the same form over the same pairs, and its skip condition adds, for each r_2 , at most one implication check per pair of a rule of S_1 and a rule of S_2 . The total is polynomial in $|S_1| + |S_2|$.

Soundness. Steps 1 and 2 return true only when S_1 accepts every invocation with no approval requirement or S_2 accepts none, and containment then holds trivially. Otherwise S_2 is a list of rules. Suppose the procedure returns true, and let I be an invocation the evaluator accepts under S_2 , decided there by the first rule $r_2 \in S_2$ that matches it. Step 3(a) exhibited the first $r_1 \in S_1$ with $r_2 \subseteq r_1$. Then r_1 matches I as well: the audience, tool and action agree, so step 1(a) of Section 6.3 passes; the parameter declarations are identical, so the closed-world check of step 1(b) passes; r_1 is well-formed (Section 6.1), so every path its clause mentions is declared and therefore present, and step 1(c) passes; and r_2 's clause holds of I and implies r_1 's, so step 1(d) passes. Hence I is accepted under S_1 . For the approval component, the rule that actually decides I under S_1 is the first rule of S_1 matching I . Because r_1 matches I , that rule is either r_1 itself, whose approval requirement step 3(a) compared with r_2 's, or a rule r' preceding r_1 . In the second case r' and r_2 both match I , so by the identity argument above they have identical declarations. They also agree on audience, tool and action, and I satisfies both clauses, so those clauses are jointly satisfiable. Nor was r' skipped. A skipping rule r'_2 would precede r_2 in S_2 , agree with r' on audience, tool, action and declaration, and have a clause implied by r' 's. So I , which matches r' , would match r'_2 too — it passes the same closed-world check, every path in r'_2 's clause is declared by well-formedness and so present, and that clause is implied by r' 's — contradicting the choice of r_2 as the first rule of S_2 matching I . Step 3(b) therefore examined r' , and its approval requirement is no stronger than r_2 's. Thus I is accepted under S_1 with an approval requirement no stronger than the one S_2 imposes, which is Definition 1. $\square$

Remark 1 (Incompleteness). The procedure is sound but not complete. Step 3(a) requires each rule of S_2 to be subsumed by a *single* rule of S_1 , so a child rule covered only by the union of several parent rules is rejected. Take S_1 to be the two rules `allow at=v tool=t action=a params {x: int} where x <= 10` and `allow at=v tool=t action=a params {x: int} where x > 10`, and S_2 the single rule `allow at=v tool=t action=a params {x: int}`. Every invocation accepted under S_2 is accepted under S_1 , so $S_2 \subseteq S_1$ holds semantically; but neither rule of S_1 subsumes S_2 's rule on its own, and the procedure returns false. The incompleteness is conservative — the procedure rejects some legitimate delegations and accepts no illegitimate ones — and we prefer that direction of error. Deciding Definition 1 exactly would require reasoning about unions

of rules, which the per-rule structure of the procedure does not support; we leave it open whether the exact relation admits an efficient decision procedure for this language.

6.5 Limitations of the Policy Language

The policy language is deliberately bounded. It cannot express semantic conditions of the form “is this email message professional?” or “is this code review constructive?” that would require external evaluation, typically by another LLM. Such conditions are non-deterministic and admit no obvious decidability or termination guarantee, and we judge them inappropriate for inclusion in authorization policies that must be cryptographically committed to via `policy_hash`.

The language also cannot express conditions referring to state outside the invocation itself: account balances, current time relative to business hours, counts of prior invocations within a window. Such conditions either require stateful evaluation (which would violate the side-effect-free requirement) or external queries (which would introduce latency and additional trust dependencies). Where stateful authorization is required, we anticipate it being handled at the tool implementation layer, after the protocol’s stateless authorization check has succeeded.

The `under` operator is lexical. Its canonical-form requirement stops `..` traversal, but it cannot see the filesystem: a symbolic link inside the permitted tree can point outside it. A tool that accepts a path authorized by `under` must therefore resolve it without following links out of the tree, and the protocol’s check does not relieve it of that.

The closed-world condition of Section 6.3 carries an operational cost of its own. Adding an optional parameter to a tool is not a backward-compatible change: every rule governing that tool must be updated to declare the new parameter before any invocation may supply it, and until the updated policy has propagated, invocations carrying the new parameter are denied. Because policies are committed to by hash in session tokens, that update is a policy version change rather than an edit in place, and every partner whose verifiers accept the policy must pin the new hash before sessions under it will verify. We consider this the right default, since the failure mode is denial rather than unauthorized acceptance, but deployments whose tool schemas change often should expect policy churn in proportion. The audience clause of Section 6.3 has the same kind of cost. Adding a verifier, such as a new region or a staging instance, means adding rules that name it. That is a new policy version, which every partner pinning the policy must re-pin, including partners the new verifier does not affect. This is the re-pinning that Section 4.5 avoids for approval services by naming them by identifier. For audiences we accept it, since which verifiers a policy authorizes is part of what the partners agreed to. An organization can confine the churn by publishing a separate policy for each partner.

These limitations are deliberate. A more expressive language would provide more flexibility but at the cost of guarantees we consider essential. We discuss possible extensions to semantic conditions via verifiable LLM attestation in Section 8.

7 Security Analysis

This section presents the security argument for DelegationChain. We state the protocol’s security properties as theorems, reduce each to standard cryptographic assumptions or to the properties of the policy language, and sketch proofs. The arguments here are informal in the sense that they have not been mechanized in a proof assistant or verified using symbolic protocol analyzers such as Tamarin or ProVerif. We discuss the planned formalization in Section 8.2.

7.1 Assumptions

The security arguments below rest on the following assumptions:

BLS unforgeability. The BLS signature scheme (Boneh, Lynn, et al., 2001; Boneh, Bradley, et al., 2026), instantiated on BLS12-381 (Sakemi et al., 2026), is existentially unforgeable under chosen-message attack, under the co-computational Diffie-Hellman assumption on the curve and in the random oracle model for HashToG₂.

BLS aggregate security. Aggregate signatures over pairwise distinct messages are existentially unforgeable (Boneh, Gentry, et al., 2003). No PPT adversary can produce a tuple $(pk_1, \dots, pk_n, m_1, \dots, m_n, \sigma_{\text{agg}})$ with the m_i pairwise distinct such that the multi-pairing check accepts, unless it has obtained valid individual signatures on each m_i from the holder of sk_i . The result holds under the same assumptions as BLS unforgeability. It places no requirement on how the pk_i were registered, nor on their being distinct. Section 4.3 guarantees distinct messages, and phase 8 of Algorithm 2 checks it.

Collision resistance of SHA-256. SHA-256 is collision-resistant in the standard sense.

Honest signing key custody. Signing keys remain in their designated hardware security modules and are not exfiltrated. Compromise of an HSM is explicitly out of scope per Section 3.4.

Honest registry root. Registry root signing keys are honestly managed. Compromise of a root is addressed via rotation and detection (Section 5.5); the formal arguments below assume an honest root, with the implication that root compromise reduces security to a recovery problem rather than a forgery problem.

Verifier configuration integrity. The set of policy hashes a verifier has pinned for each partner, and the root key it associates with each partner organization, reflect its own organization’s decisions and cannot be written by the adversary. The first link of Eq. (6), and through it Theorems 4 and 6, is only as good as that configuration.

7.2 Theorems

Theorem 1 (Token unforgeability). *Under BLS unforgeability, no PPT adversary can produce a session, delegation, or invocation token that verifies under a public key pk for which it does not hold the corresponding sk , except with negligible probability.*

Proof sketch. A valid token consists of a body and a BLS signature on the body’s domain-tagged hash. Forging the signature without sk contradicts BLS unforgeability under chosen-message attack: any adversary that forges a token can be used to construct an adversary against BLS unforgeability by treating the domain-tagged hash as the message and extracting the signature. $\square$

Theorem 2 (Parameter binding). *A valid invocation token for parameters P cannot be repurposed as a valid invocation token for parameters $P' \neq P$, except with negligible probability.*

Proof sketch. The signed message m_N contains $\text{Canon}(\text{InvocationBody})$, which includes $\text{Canon}(P)$. Under collision resistance of SHA-256 and deterministic encoding, $\text{Canon}(P) \neq \text{Canon}(P')$ implies $m_N \neq m'_N$. The signature σ_N valid on m_N is invalid on m'_N by BLS unforgeability. The redundant `params_hash` field provides defense-in-depth against canonical encoding bugs but is not required for the theorem. $\square$

Theorem 3 (Chain integrity). *The aggregate signature over a delegation chain covers the entire chain content: any modification, reordering, truncation, or extension of the chain invalidates the aggregate, except with negligible probability.*

Proof sketch. Modification of any body changes its canonical encoding, changing the corresponding m_k (since m_k hashes the body). Reordering changes the sequence of m_k values (since m_k commits to m_{k-1}). Truncation is caught by the same recursion. Suppose the adversary drops B_k for some interior $0 < k < N$. The verifier recomputes the remaining messages from the bodies it actually receives, so where A_{k+1} signed $m_{k+1} = H(\text{TAG_DEL} \parallel m_k \parallel \text{Canon}(B_{k+1}))$ the verifier computes $m'_{k+1} = H(\text{TAG_DEL} \parallel m_{k-1} \parallel \text{Canon}(B_{k+1}))$. Since $m_k \neq m_{k-1}$ except with negligible probability under collision resistance, $m'_{k+1} \neq m_{k+1}$, and every subsequent message differs likewise. The adversary would need signatures under the surviving hops' keys on messages those hops never signed, which contradicts BLS aggregate security. Dropping the first or last hop is caught earlier still, by the position-determined tag check: a chain whose leading token is not a session token, or whose trailing token is not an invocation token, is rejected in phase 1 of Algorithm 1. No assumption about the secrecy of individual σ_k values is required, and we make none — the accumulation scheme of Section 4.3 would not support one. Extension requires producing a new σ for the extension hop; under BLS aggregate security, this requires the secret key of the extending party. The position-determined tag check (Section 4.3) rejects chains where an invocation token is repurposed as a delegation token (or vice versa) for chain extension. $\square$

Theorem 4 (Scope monotonicity). *In any chain accepted by the verifier, every link of the containment chain Eq. (6) holds in the sense of Definition 1, where S is a policy the verifier has pinned for the issuing organization. In particular, the session scope is contained in S , and the scope at hop $k + 1$ is contained in the scope at hop k .*

Proof sketch. Phase 6 of Algorithm 2 rejects any session whose policy hash is not pinned for the issuing organization. That organization is $\text{org}(\text{sid}(B_0))$, read from the signed session body. Phase 5 of Algorithm 1 has already required the issuer's certificate to verify under the root the verifier holds for that organization, and to name it as its registry. So the pins consulted are those of the organization that actually issued the session, and a registry trusted only for some other organization cannot redirect the lookup. Phase 5 has also required that certificate to be of kind **issuer**. So the session scope is one the organization's issuance service granted, not one an agent or approver granted itself. Phase 6 then resolves S by content address, so by collision resistance of SHA-256 S is the document the pinned hash names. Finally it checks each link of Eq. (6) explicitly, using the containment procedure of Section 6.4. Acceptance of the chain implies that every one of these checks returned true, and by the soundness half of Proposition 2 that implies containment in the sense of Definition 1. The converse does not hold: by Remark 1 the verifier will also reject some chains whose scopes are genuinely nested, so the guarantee is that accepted chains are monotone, not that all monotone chains are accepted. The integrity of the body content from which the policy hash and the scopes are read is guaranteed by Theorem 3. The first link is only as strong as the verifier's pinned set, which is an assumption (Section 7.1) and not something the protocol establishes. $\square$

Theorem 5 (Replay resistance). *Consider a single logical verifier maintaining a correctly-sized nonce cache with an atomic insert-if-absent operation — one cache shared across all instances that accept chains, rather than an independent cache per replica — that accepts a chain only when inserting its nonce-and-invoker-key pair succeeds. It accepts at most one chain for each such pair within the validity window, however requests are interleaved, and rejects every chain presented after the validity window has expired. Moreover, a chain addressed to one verifier is rejected by every verifier with a different identifier.*

Proof sketch. Acceptance requires a successful insertion of the chain’s (pk_N, nonce) pair. The insertion is atomic, so of any number of chains carrying the same pair, whether presented one after another or concurrently, at most one insertion succeeds while the pair remains in the cache; the phase-3 lookup only rejects some replays earlier. The cache TTL is the validity window plus a tolerance at least as large as the clock disagreement among the instances sharing the cache, so the pair is not evicted while a replay could still pass the temporal checks at any of them, and outside the window the verifier rejects on the expiry check before any signature verification. For the second part, the invocation body names its verifier in **aud**, which is part of m_N . Phase 1 rejects a chain whose **aud** is not the verifier’s own identifier, and altering **aud** changes m_N and invalidates the aggregate (Theorem 3). Distinct verifiers therefore need no shared cache, provided they have distinct identifiers. $\square$

Theorem 6 (Approval binding). *An approval receipt for invocation I cannot be reused for any invocation $I' \neq I$, and an invocation requiring approval from service s cannot be accepted with approval from service $s' \neq s$, except with negligible probability.*

Proof sketch. The approval body signed by the approval service includes the invocation digest of Section 4.5, a tagged hash of the invocation body without its receipts. Modification of the invocation changes this digest, invalidating the receipt under BLS unforgeability. Phase 7 checks each receipt under the key the registry currently certifies for the service it names, with kind **approver**, so a receipt signed by any other key, or under a revoked certificate, is rejected. The required approval service identifiers are named by the policy S . The session names S by hash, but the verifier accepts the hash only if it has pinned it for the issuing organization. S is therefore a policy the receiving organization agreed to, not one the issuer was free to choose. Resolution is content-addressed (Section 5.4), so no other document can be substituted behind the hash.

The requirement then propagates inward. By Theorem 4, every link of the containment chain Eq. (6) holds, and by the approval clause of Definition 1 each containment can only strengthen the approval requirement. The requirement the verifier obtains by evaluating the invocation against $\text{scope}(B_{N-1})$ is therefore no weaker than the one S imposes, even though the intermediate scopes were written by the agents themselves. Routing the approval elsewhere disturbs none of this: the policy hash is fixed at session start and does not vary with the agent’s routing choice, so a misrouted chain arrives carrying the correct hash and the wrong receipt, and phase 7 of Algorithm 2 rejects it because no receipt signed by a required service is present. $\square$

7.3 Threats Addressed

The five prevented threat families of Section 3.3.1 are addressed by the above theorems and the identity infrastructure of Section 5, as follows. Token forgery (T1a) is addressed by Theorem 1, and chain forgery (T1b) by Theorem 3. Parameter substitution (T2a) is addressed by Theorem 2, and scope escalation (T2b) by Theorems 3 and 4 together. Cross-session replay (T3a) and intra-window replay (T3b) are addressed by Theorem 5, under the single-logical-verifier hypothesis stated there; cross-context token reuse (T3c) is addressed by domain separation tags (Section 4.3); and cross-verifier reuse (T3d) is addressed by the audience check of phase 1, the second part of Theorem 5. The mandatory audience clause of every rule (Section 6.3) goes further, and lets a policy state which verifiers it authorizes at all. The prevented part of the approval family (T4b–T4d) is addressed by Theorem 6 together with the policy containment chain of Eq. (6); T4a is bounded rather than prevented and is discussed below. The prevented part of the identity family (T5a, T5c, T5d) is addressed by proof-of-possession (Section 5.3), short-lived certificates and revocation (Section 5.6), and optional transparency log integration. Certificate substitution (T5d) additionally relies on the namespace binding of Section 5.4: a certificate must verify under the root the verifier holds for the organization its identifier names. Role confusion (T5e) is

addressed by the kind checks of phases 5 and 7. They accept only an **issuer** certificate for the session body, only **agent** certificates for later hops, and only **approver** certificates for receipts. Pinning adds an informal bound outside the honest-root assumption. An issuer whose keys have been compromised can still grant sessions only within the policies the verifier has pinned for that issuer’s own organization, and because of the kind check a compromised agent or approver key cannot act as an issuer at all. The qualifier depends on the namespace binding. Without it, a compromise in one organization could draw on the policies pinned for another.

The threats of Section 3.3.2 are bounded, not prevented. A *compromised approval service* (T4a) can issue receipts the human approver never authorized, and no check in Algorithms 1 and 2 can distinguish them from honest ones. The bound comes from the decoupling of Section 4.5: because approval signatures are verified separately from the chain aggregate, the compromise cannot extend to chain forgery or scope escalation, and it reaches only those invocations that policy routes through the affected service. A *compromised registry root* (T5b) can certify any identity in its own organization’s namespace. Its reach is bounded by the namespace binding of Section 5.4, which keeps it out of every other organization’s identifiers, and by pinning: the sessions it enables stay within the policies verifiers have pinned for that organization. *Within-policy prompt injection* (T6a) — an adversary inducing an agent to take an action that is within its permitted policy but contrary to the principal’s intent — is not prevented by the protocol. The protocol’s contribution is to *bound the blast radius*: the adversary can cause only invocations that policy already permits and, where policy requires human approval, will surface the action to an approver who has an opportunity to refuse. Tightly-scoped policies and appropriately-required approvals reduce the blast radius; defending against prompt injection itself remains an active research area outside the protocol’s purview. *Confused-deputy delegation* (T6b), where an agent is induced to delegate authority to an adversary, is similarly bounded rather than prevented: the agent cannot delegate more authority than it itself holds (Theorem 4), limiting damage to the agent’s own permitted scope.

7.4 Limitations of the Analysis

We emphasize that the analysis above consists of proof sketches under specified assumptions, not machine-checked or peer-reviewed proofs. Several aspects warrant scrutiny in any subsequent formalization. Theorem 3 does not depend on proof-of-possession. It rests on the messages of a chain being pairwise distinct, which the digest recursion guarantees except with negligible probability and which phase 8 of Algorithm 2 also checks explicitly. Proof-of-possession remains a registration requirement, but a failure to enforce it would affect attribution (T5a), not chain integrity. Theorem 5 depends on operational properties of the nonce cache (correct sizing, crash recovery, distributed consistency across verifier replicas) that are not enforced by the protocol but must be discharged by the verifier implementation. The containment procedure of Section 6.4 is sound but not complete with respect to Definition 1 (Remark 1); its soundness has been argued informally here and would repay mechanization, since it is the step on which Theorem 4 rests. Symbolic verification of the protocol in Tamarin or ProVerif would confirm or refute these arguments and is planned future work.

8 Discussion

This section discusses related work, limitations of the protocol and its analysis, and directions for future work.

8.1 Related Work

DelegationChain draws on three distinct research traditions: capability-based authorization, pairing-based cryptographic protocols, and agent-system security. We position our contribution relative to each.

Capability-based authorization. The conceptual framework of attenuated capabilities, in which authority is represented by unforgeable tokens that can be progressively restricted as they are delegated, dates to early work on capability operating systems and was formalized in the modern context by Miller (2006). The most directly relevant prior work is Macaroons (Birgisson et al., 2014), which applied attenuated capabilities to cloud authorization using HMAC chains. A Macaroon carries “caveats,” predicates that restrict its use; each delegation appends additional caveats, and verification recomputes the HMAC chain using a shared secret known to the verifier. Macaroons demonstrated that attenuated delegation could be made cryptographic and efficient. Macaroons also provide *third-party caveats*, in which a caveat names a principal other than the target service and is satisfied by a discharge macaroon obtained from that principal (Section 2.3). This is the mechanism by which Macaroons reach across trust boundaries, and any comparison that treats Macaroons as a single-domain-only design is inaccurate. DelegationChain extends the model in three principal directions: from shared-secret HMAC to public-key BLS signatures; from per-hop linear signature growth to compact aggregate signatures via BLS aggregation; and from informal caveat languages to a typed declarative policy language with termination and containment guarantees.

The substantive difference from third-party caveats is the shape of the verification path rather than its reach. Discharging a third-party caveat requires the holder to contact the named principal and collect a discharge macaroon before presenting the token; the third party must be available at that moment, and each additional third party adds a round trip. DelegationChain moves that authority into signatures that the verifier checks locally against registry-resolved public keys, so the request path carries no discharge-collection step. The cost of this choice is the identity infrastructure of Section 5, which Macaroons does not require.

UCAN (UCAN Working Group, 2026) also extends capability delegation to a public-key setting, originally for Web3 contexts. UCAN 1.0 signs tokens canonically encoded in DAG-CBOR, inside a Varsig-headed envelope, with Ed25519 recommended and P-256 and secp256k1 also required, and expresses delegation attenuation through scoped capability tokens. Our work differs from UCAN in several respects: we use BLS signatures specifically for their aggregation property, which UCAN does not provide; UCAN 1.0 does specify a policy language over command arguments, but we did not find a defined containment relation between capabilities, which is what Section 6.4 supplies and what the scope monotonicity check requires; we target MCP rather than Web3 transport; and we include a federated identity infrastructure with proof-of-possession and revocation semantics adapted to the short-lived-credential model.

Authorization certificates and delegation chains. The problem of composing delegation chains and computing the authority that survives composition is older than this literature. SPKI/SDSI (Ellison et al., 1999) defines authorization certificates in which the principal is a public key rather than a name, and specifies a *5-tuple reduction* algorithm that composes a chain of delegation certificates into a single effective authorization by intersecting the authorization tags at each step, subject to a delegation bit controlling whether further delegation is permitted. Reduction is the direct ancestor of the scope-monotonicity check of Section 6.4, and the correspondence is close: SPKI intersects tags where we require containment between scope expressions. The differences are that SPKI computes the intersection, producing the resulting authority, where our verifier only checks a containment relation that the delegator has already

asserted; that SPKI's tag **forms* (prefix, range, set) are a fixed and deliberately small constraint vocabulary, where our policy language is typed and admits conditions over named parameter paths; and that SPKI has no notion of signature aggregation.

X.509 proxy certificates (Tuecke et al., 2004) apply the same idea within Web PKI and were deployed at scale in grid computing. An end-entity certificate signs a short-lived proxy certificate, which may itself sign another, producing a delegation chain that a relying party validates as an ordinary X.509 path; attenuation is expressed through an embedded `ProxyCertInfo` policy field. This is the closest classical prior art to the certificate-chain-plus-policy design of Section 5. Two differences are worth naming. RFC 3820 leaves the content of the policy field opaque — it names a policy language by OID and carries its bytes, without specifying one — so nothing in the standard provides the decidable containment check our Section 6 is built around. And proxy certificate chains root in a CA hierarchy, where our registry model is federated and explicitly declines a global trust anchor (Section 5.1).

Biscuit (Eclipse Biscuit Project, 2026) is the closest contemporary system in the capability-token line. Biscuit tokens are public-key verifiable, support offline attenuation — a holder may append a block of additional checks at any time, restricting the token without contacting the issuer, and cannot remove a block without invalidating the signature — and carry an authorization language based on a purpose-built Datalog with specified evaluation semantics. Biscuit's Datalog is more expressive than our policy language, and its formal treatment of evaluation is more developed than the sketch we give. The differences run the other way on two axes: Biscuit signs each block individually with Ed25519 or ECDSA and does not aggregate, so token size grows with attenuation depth; and Biscuit does not bind a token to a specific invocation's parameter values, since attenuation constrains future use rather than committing to one call. We regard Biscuit's Datalog as the more serious policy design and note that a containment check over it would be a substantially harder problem than the one Proposition 2 addresses.

ZCAP-LD (Lemmer-Webber et al., 2026), developed in the W3C Credentials Community Group, expresses object capabilities as linked-data documents with invocation and delegation proofs, and shares the attenuated-delegation model. Its current draft attenuates in three fixed ways. A delegated capability may append a path or query suffix to its parent's invocation target, must carry an expiry, and may allow only actions its parent allowed. That vocabulary is small and its containment is straightforward: a prefix check, a date comparison and a subset test. ZCAP is therefore not outside the decidability constraints of Section 6, as an earlier draft with extensible caveats would have been. What it lacks is any way to constrain an invocation's arguments beyond a prefix of the target URL, with no range or membership conditions of the kind Section 6 provides. It also remains a community-group draft rather than a standard.

Proof-carrying authorization. A separate tradition places the burden of justifying access on the requester. Appel and Felten (1999) introduced proof-carrying authentication, in which a request arrives accompanied by a proof, in a higher-order logic, that it is authorized; the verifier need only check the proof, and the framework is expressive enough to encode Taos, SPKI, SDSI and X.509. Bauer (2003) developed the approach into a deployed system for web access control. The comparison is instructive because the trust structure is the same as ours — the verifier checks locally, without consulting the issuer — while the mechanism is the opposite: proof-carrying authorization gains expressiveness by admitting an arbitrary logic and pushing proof search to the requester, where we give up expressiveness (Section 6.5) to keep the verifier's work bounded by a syntactic check. A `DelegationChain` chain can be read as a very restricted proof, in which the only inference rules are signature verification and scope containment.

Cryptographic infrastructure. The BLS signature scheme is due to Boneh, Lynn, et al. (2001), and aggregation over distinct messages, with its security proof, to Boneh, Gentry, et al.

(2003); that proof is the one our chain integrity argument uses. Rogue-key defenses for the same-message setting — proof-of-possession (Ristenpart and Yilek, 2007) and the alternative of Boneh, Drijvers, et al. (2018) — are not needed for it, and we use proof-of-possession only for attribution (Section 5.3). We use the signing variant standardized by the CFRG (Boneh, Bradley, et al., 2026), the BLS12-381 curve parameters of Sakemi et al. (2026), and the hash-to-curve construction of RFC 9380 (Faz-Hernández et al., 2023).

The problem our chain digest solves — producing a single compact signature over an *ordered* sequence of signers on distinct messages, such that the order is itself authenticated — is the problem that *sequential aggregate signatures* were introduced to solve. Lysyanskaya et al. (2004) gave the first construction, in which each signer folds its signature into the aggregate it receives from its predecessor, using certified trapdoor permutations; the order of signing is bound by the construction itself. Boldyreva et al. (2007) extended the line to ordered multisignatures and identity-based sequential aggregates in the pairing setting. We should therefore be precise about what is and is not new here. Ordered, aggregated signature chains are not new, and we do not claim them.

What we do claim is narrower: a specification of how to obtain order binding for delegation chains using a recursive hash chain over domain-separated token bodies together with *plain* BLS aggregation, rather than a dedicated sequential aggregate scheme. The reason for the choice is practical rather than theoretical. The construction needs no trapdoor permutations and no scheme-specific sequential signing operation; it uses the ordinary signing and aggregation operations of the CFRG BLS draft (Boneh, Bradley, et al., 2026), so it can be built on any conforming library, and the order binding lives in data the verifier already has to hash. What we give up is that the hash chain must be recomputed by the verifier from the received bodies, whereas a native sequential aggregate carries order binding inside the signature object. Whether that trade is worth making is an empirical question about verification cost, which this paper does not settle. Certificate Transparency (Laurie et al., 2021) provided the model for our optional transparency log integration.

MCP and agent-system security. The security gap in cross-organizational MCP deployments has been empirically characterized by recent measurement work. H. Zhou et al. (2026) surveyed 7,973 live remote MCP servers and found 40.55% exposing tools without authentication; of the OAuth-enabled servers they were able to probe actively, every one exhibited at least one authentication flaw, and the disclosure process yielded nine CVE identifiers. Single-organization runtime enforcement approaches (Bühler et al., 2026) provide defense-in-depth within an organization’s perimeter but do not provide cross-organizational cryptographic proofs and are not designed to extend across organizational boundaries.

A second group of proposals secures the other direction of the MCP relationship. Maloyan and Namiot (2026) give a security analysis of the MCP specification and propose a backward-compatible extension — called MCPSec in their abstract and AttestMCP in the body of the paper — in which servers prove their claimed capabilities with certificates from a capability authority, and JSON-RPC messages carry HMAC authentication bound to server identity. Metere (2026) similarly lets a host decide which tool servers, and which of their tools, it will admit. Both protect the host from untrustworthy servers. DelegationChain protects the server from unauthorized callers, so the two directions are complementary.

Closer to our direction, ACLE-MCP (Ding et al., 2026) addresses what its authors call the post-authorization execution trust gap: OAuth authorization does not ensure that a later call is executed by the workload the relying party meant to trust. For each protected call it issues a short-lived, sender-constrained capability lease binding the expected workload, the operation, object and parameter bounds, and downstream constraints; a provider-side gate consumes the lease before the tool runs. The differences are of mechanism and of scope. ACLE-MCP’s

lease carries parameter bounds set when the lease is issued; our invocation token binds exact values, which the verifier then checks against the policy’s conditions. Its leases are issued per call, where our delegation prefix is built once and only the invocation token is signed per call, without contacting any issuer. Conversely, ACLE-MCP attests the executing workload, which DelegationChain does not address at all.

Llambí-Morillas and Fernández-Fernández (2026) formalize cryptographically verifiable agent authorization as a relation binding an agent principal, a concrete request, an execution context and the satisfaction of a policy, with a zero-knowledge proof of concept. Their request binding corresponds closely to what we call parameter binding. Their separation of request binding from runtime execution binding names the gap that ACLE-MCP addresses and that we leave open. Z. Zhou (2026) proposes governance requirements for agents that act across organizational boundaries — capability integrity, behavioral verifiability and interaction auditability. Among its results is that one unverifiable interior agent breaks end-to-end verification for every downstream node, the multi-agent counterpart of our requirement that every hop of a chain verify. Binding authorization to an invocation’s parameter values is not among its requirements.

The closest contemporary work is the Agent Identity Protocol of Prakash (2026), which proposes invocation-bound capability tokens for delegation across MCP and A2A. AIP has two wire formats. Its compact mode, for single-hop cases, is an EdDSA-signed JWT carrying issuer, subject, scope, a budget, a maximum delegation depth and timestamps. Its chained mode is a Biscuit token with Datalog policies, in which each delegating agent appends a signed block that attenuates scope. Verification is local, with no authorization-server round trip. Identifiers either resolve over HTTPS (`aip:web`) or are the public key itself (`aip:key`), which is the same shape as our design. The authors list identity resolution, protocol bindings, completion blocks recording provenance, policy profiles and the compact mode as what AIP adds to Biscuit, and they ship interoperable Rust and Python implementations.

Three differences matter. First, despite the name, neither AIP format signs the parameter values of an invocation. Scopes name capabilities such as a tool, and — as with Biscuit, of which the chained mode is an instance — attenuation constrains future use rather than committing to a particular call. Parameter binding (Section 4.4) is the property on which the two designs differ most. Second, AIP signs each hop individually, where we aggregate. Third, AIP has no registration step with proof-of-possession and no federation model, and it defers revocation to a later version, relying for now on short-lived tokens; Section 5 supplies all three. In the other direction, AIP has working implementations and published performance measurements, which this paper does not. It is the first baseline our evaluation must be compared against (Section 8.3).

Workload and identity infrastructure. SPIFFE/SPIRE (SPIFFE Project, 2026) provides workload identity through short-lived X.509 certificates and JWT-format SVIDs. While SPIFFE addresses workload identity within a trust domain, it does not provide parameter-bound authorization, delegation chain integrity, or aggregate signatures. OAuth 2.1 (Hardt et al., 2026) addresses caller identity and scope, may require synchronous token introspection on the verification path, and does not provide parameter binding.

For crossing organizational boundaries, the OAuth working group’s identity-chaining draft (Schwenkschuster et al., 2026) combines token exchange with the JWT authorization grant. The client’s authorization server in one domain issues a signed grant, which the authorization server in the next domain validates and exchanges for an access token, and the pattern repeats at each boundary. Scope can be narrowed at each exchange, so attenuated delegation across organizations is expressible in OAuth. The differences from DelegationChain are structural. Every boundary costs round trips to authorization servers on both sides. What reaches the resource is a token from its own domain’s authorization server, which vouches for the earlier hops rather than letting the resource verify them. And nothing binds the token to one invocation’s argument

Table 1: DelegationChain against related systems. ✓ provided; p provided with qualifications (see notes); – not provided; u not specified by the cited source. “No anchor” abbreviates cross-organizational verification without a shared trust anchor.

System	No anchor	Param. bind	Aggregat.	Offline	Signature scheme	Attenuation	Policy formalization
DelegationChain	✓	✓	✓	p	BLS12-381, aggregate	scope containment	typed grammar; termination proof; sound containment check
Macaroons (Birgisson et al., 2014)	p	✓	–	p	HMAC chain	caveat append	caveats, informal
UCAN (UCAN Working Group, 2026)	✓	✓	–	✓	EdDSA / ECDSA	capability attenuation	policy language specified; no containment relation
SPKI/SDSI (Ellison et al., 1999)	✓	p	–	✓	scheme-independent	5-tuple reduction	tag *-forms, specified
X.509 proxy certs (Tuecke et al., 2004)	–	u	–	p	X.509 (RSA / ECDSA)	embedded policy field	policy field opaque
Biscuit (Eclipse Biscuit Project, 2026)	✓	–	–	✓	Ed25519 / ECDSA-P256	offline block append	Datalog, specified semantics
SPIFFE (SPIFFE Project, 2026)	p	–	–	✓	X.509 / JWS	–	n/a (identity only)
OAuth 2.1 (Hardt et al., 2026)	–	–	–	p	JWS (bearer)	–	scopes, informal
AIP (Prakash, 2026)	✓	–	–	p	EdDSA JWT / Biscuit	capability attenuation	Datalog, via Biscuit

values. We view DelegationChain not as a replacement for these systems but as complementary infrastructure for the specific use case of verifiable agent-to-tool invocation across organizations.

Summary comparison. Table 1 collects the comparisons made above. The columns are the four properties this paper claims to combine (Section 9), plus the signature scheme, the attenuation mechanism, and the degree to which the policy component is formally specified. We have marked a cell u wherever the cited source does not settle the question, rather than infer an answer from an implementation. The reader should note that our own row is not a column of ✓: offline verification is qualified for DelegationChain as it is for several others, for the reasons Section 8.2 gives.

Notes on the qualified cells. Macaroons is p on *no anchor* because the target service must hold its own root secret, while third-party caveats require no secret shared with the third party; it is p on *offline* for the same reason, since discharging a third-party caveat costs a round trip. SPKI/SDSI is p on *parameter binding* because authorization tags constrain values through *-forms without binding a certificate to one invocation’s exact parameters. X.509 proxy certificates are u on parameter binding because RFC 3820 names a policy language by OID and carries its bytes without specifying one, and p on offline because path validation is local while revocation checking is not. Biscuit is – on parameter binding because attenuation constrains future use rather than committing to a single call. SPIFFE is p on *no anchor* because cross-domain use requires SPIFFE Federation. OAuth 2.1 is p on offline because a JWS access token verifies locally while introspection does not; the row describes the OAuth 2.1 core, and cross-domain chaining is discussed in Section 8.1. AIP is – on parameter binding because neither of its formats signs an invocation’s argument values, and p on offline because verification is local but `aip:web` identifiers resolve over HTTPS. DelegationChain is p on offline for the reasons given in Section 8.2.

8.2 Limitations

We discuss limitations of the work along several dimensions.

Analytical, not empirical. This paper presents a protocol design and preliminary security analysis. It does not present empirical performance measurements from a reference implementation. The verification cost analysis of Section 4.6 is based on counting cryptographic operations and library-reported costs for those operations; precise wall-clock latency in the deployment configurations of interest will depend on implementation choices (BLS library, multi-pairing strategy, deserialization performance) that this paper does not commit to. A Rust reference implementation using the `blst` library is under development, and the empirical evaluation described in Section 8.3 is in progress.

Offline verification is qualified. We claim verification that requires no synchronous call to the issuing authority *on the dominant path*, and the qualification is load bearing. Two dependencies remain, and a third can be avoided. Certificate resolution (Section 5.4) queries the issuing registry on a cache miss. Revocation (Section 5.6) requires polling, push, or log monitoring, which is a continuous background dependency rather than a per-request one. Policy resolution is the avoidable one. Pinned policies can be fetched when they are pinned (Section 5.4), and only a verifier that defers the fetch to first use pays a round trip, on that call. In the steady state — a partner whose certificates and policies are already cached — verification touches none of these and is genuinely local. On a cold cache it is not, and statements elsewhere in this paper about offline verification should be read with that scope.

The obvious repair is to carry certificates in the chain, as X.509 does, so that a verifier needs the registry root key but not the registry. This would make verification offline modulo revocation, and we note it as a deployment option rather than adopting it, because the arithmetic is unfavourable in exactly the dimension the design optimizes. A certificate carries a 48-byte public key and a 96-byte registry signature before any identifier, timestamp, or serial-number field, so attaching one per hop necessarily costs more per hop than the 96 bytes that aggregation saves across the whole chain. A design that carried certificates inline would be giving up the bandwidth argument for aggregation in order to buy offline verification, and it is not obvious which of the two a deployment should prefer. Settling it requires the measurements described in Section 8.3, not further argument here.

Security arguments are sketches, not proofs. Section 7 presents proof sketches that reduce protocol security to standard cryptographic assumptions. These sketches have not been mechanized in a proof assistant nor have they been verified by independent cryptographer review. Symbolic protocol verification using Tamarin or ProVerif is planned and is expected to reveal at minimum the kind of subtle interaction issues that hand-analysis typically misses (e.g., key-compromise impersonation paths, unknown key-share variants). We treat the current analysis as a starting point for formal verification, not as a substitute for it.

Policy language expressiveness. The policy language of Section 6 is deliberately bounded. It cannot express semantic conditions requiring evaluation by an LLM or external system (“is this email professional?”), stateful conditions referring to data outside the invocation (“is this user under their monthly limit?”), or conditions requiring synchronous queries to external systems. These limitations are intentional: each relaxation would compromise the decidability, termination, or side-effect-freedom guarantees on which the security analysis depends. Where richer policy expressions are required in practice, we anticipate them being enforced at the tool implementation layer *after* the protocol’s stateless check has succeeded. A fully-integrated treatment of stateful

policy may require extensions to the protocol — including verifiable LLM attestation for semantic conditions — which we leave as future work.

Threat model boundaries. Several classes of threats are explicitly out of scope (Section 3.4), including fully-compromised HSMs, compromised signing service infrastructure, side-channel attacks, and attacks on human approvers’ devices. The protocol provides no defense against these and relies on operational measures (HSM hardening, key custody discipline, secure approver workflows) for mitigation. Within-policy prompt injection is bounded but not prevented (Section 7.3); the protocol’s contribution is to ensure that prompt-injected agents cannot exceed their permitted policy, not to prevent prompt injection itself.

Post-quantum security. BLS12-381 is not post-quantum secure. A sufficiently capable quantum adversary could forge BLS signatures. Migration to a post-quantum signature scheme is a future protocol version; this likely involves hybrid construction combining BLS with a post-quantum scheme during a transition period, with the complexity tradeoffs that hybrid constructions imply.

Bootstrapping and adoption. The federated registry model requires both communicating organizations to operate compatible registries (or to participate in a shared transparency log). Adoption of the protocol in cross-organizational agent ecosystems thus faces a bootstrapping problem: early adopters have limited peers to communicate with, and the value of adoption grows with peer density. We do not address bootstrapping strategies in this paper, but observe that the same dynamic affects all cross-organizational identity protocols, including SPIFFE and SAML federation. Successful adoption is likely to require either integration into a widely-deployed platform (e.g., as a feature of an MCP-compliant agent framework) or coordination through standards bodies.

8.3 Future Work

We identify several directions for future work, in approximate order of priority.

Reference implementation and empirical evaluation. A Rust reference implementation using `blst` is the most immediate next step. Empirical evaluation should measure end-to-end verification latency across chain lengths, throughput under load, and storage overhead for nonce caches and certificate caches. The first baseline should be AIP (Prakash, 2026), which addresses the same problem and has published Rust and Python implementations; comparison with OAuth token introspection and SPIFFE SVID verification on representative workloads should follow. Each comparison should be run both with and without caching of already-verified delegation prefixes, which the UCAN specification recommends for its own tokens (UCAN Working Group, 2026), because caching changes the per-call cost of every design and a comparison without it would not reflect how verifiers are built.

Symbolic verification. Modeling the protocol in Tamarin or ProVerif and proving the security properties of Section 7 will validate (or, more usefully, falsify) the hand-written proof sketches. This is expected to surface protocol details requiring revision before publication of a normative specification.

Differential privacy on audit trails. The protocol generates detailed audit records of every accepted invocation. For deployments in regulated industries, the ability to produce compliance attestations (“ $X\%$ of invocations were authorized”) without exposing the content of individual invocations would have practical value. Standard differential privacy mechanisms apply; their

integration into the protocol’s audit format is straightforward but was deferred from this work for scope reasons.

Verifiable LLM attestation for semantic policy conditions. The most significant restriction of the policy language is its inability to express semantic conditions. Recent work on verifiable LLM inference (zkML and related approaches) may enable a future extension in which semantic policy conditions are evaluated by an LLM whose evaluation is itself cryptographically attested. This is speculative; the practicality of zkML at the latencies required for authorization protocols is not currently established.

Post-quantum migration. As discussed above, planning for migration to post-quantum signatures is appropriate even before the threat is acute. Investigating hybrid BLS-plus-post-quantum constructions adapted to the aggregation property is an open question.

Cross-registry naming and discovery. This paper does not address how organizations choose and namespace their `organization-id` values. In practice deployments will use DNS-style hierarchical naming or other conventions, but standardizing this is needed before adoption at scale.

9 Conclusion

The growing deployment of LLM-based agents across organizational boundaries creates an authorization problem that existing protocols do not adequately address. Receiving organizations need cryptographic assurance that an incoming tool invocation was authorized by a specific agent operating within a specific scope on behalf of specific principals, with specific parameter values — and they need this assurance without depending on the sending organization’s infrastructure being trustworthy at verification time, and, so far as possible, without depending on its being reachable either.

We have presented DelegationChain, a protocol combining cross-organizational verification without a shared trust anchor, parameter-bound authorization, and aggregatable proof chains, with verification that is local in the steady state. That last property is qualified rather than absolute: certificate resolution touches the issuing registry on a cache miss, policy resolution does too unless policies are fetched when they are pinned, and revocation is a standing background dependency (Section 8.2). The protocol’s cryptographic core is a recursive chain digest construction over BLS12-381 aggregate signatures over pairwise distinct messages, whose security needs no assumption about how keys were registered; proof-of-possession at registration protects attribution. The associated policy expression language is terminating and side-effect-free by construction, and admits a polynomial-time containment check that is sound with respect to its semantic subset relation, supporting the verifier’s scope monotonicity check without introducing computational or semantic hazards. The federated identity infrastructure permits interoperation between mutually distrusting organizations through direct peering, federated transparency logs, or hierarchical attestation, as deployment context requires.

We have stated and sketched proofs of the protocol’s core security properties — unforgeability, parameter binding, chain integrity, scope monotonicity, replay resistance, and approval binding — and mapped these to the threat model. We have been explicit about the threats the protocol does not address, particularly within-policy prompt injection, which the protocol bounds rather than prevents.

The work has clear limitations: the analysis is hand-written rather than mechanically verified, no empirical performance data is yet available, and several threat classes remain out of scope. A reference implementation in Rust, symbolic verification in Tamarin or ProVerif, and empirical evaluation are the immediate next steps. These additions would establish DelegationChain as a viable foundation for the inter-agent authorization layer in production deployments.

We view this paper as a starting point. The design space for cross-organizational agent authorization is still being mapped, and multiple competing proposals — AIP and AgentBound among them — are appearing simultaneously with this work. We expect useful integration and competition between these proposals over the next several years as the inter-agent ecosystem matures. Our contribution is one design point in this space, optimized for combining attenuated capability semantics, public-key cross-organizational verification, and aggregate-signature efficiency. Whether this combination proves to be the right one for the deployments that emerge is a question only the deployments will answer.

Acknowledgments

The author thanks colleagues at EKTU for early discussion of the protocol’s design, and the broader MCP and agent security research communities for the context in which this work was undertaken.

Use of AI Tools

The author used a large language model (Anthropic Claude) extensively in preparing this manuscript: to draft and organize text, formalize the policy language grammar, and prepare proof sketches. The model was also used as a critical reviewer across several revision passes, in which it identified technical and citation errors and proposed corrections, alongside review comments the author obtained from other sources. Some of the resulting corrections were adopted substantially as proposed, including the skip condition of the containment procedure (Section 6.4) and the policy audience clause (Section 6.3), and the model carried out the edits under the author’s direction. All choices among design alternatives, and the threat model, are the author’s. References were verified against original sources with the model’s assistance, including whether each supports the specific claim it is cited for. The author takes full responsibility for the content, accuracy, and conclusions of this work.

References

- Appel, Andrew W. and Edward W. Felten (1999). “Proof-Carrying Authentication”. In: *Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS)*. ACM, pp. 52–62. DOI: [10.1145/319709.319718](https://doi.org/10.1145/319709.319718).
- Bauer, Lujo (Nov. 2003). “Access Control for the Web via Proof-Carrying Authorization”. PhD thesis. Princeton, New Jersey: Princeton University.
- Birgisson, Arnar, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentczner (2014). “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud”. In: *Network and Distributed System Security Symposium (NDSS)*. Internet Society.
- Boldyreva, Alexandra, Craig Gentry, Adam O’Neill, and Dae Hyun Yum (2007). “Ordered Multisignatures and Identity-Based Sequential Aggregate Signatures, with Applications to Secure Routing”. In: *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*. ACM, pp. 276–285. DOI: [10.1145/1315245.1315280](https://doi.org/10.1145/1315245.1315280).
- Boneh, Dan, John Bradley, Sergey Gorbunov, Riad S. Wahby, Hoeteck Wee, Christopher A. Wood, and Zhenfei Zhang (July 2026). *BLS Signatures*. Internet-Draft draft-irtf-cfrg-bls-signature-07. Work in progress. URL: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-bls-signature/07/>.

- Boneh, Dan, Manu Drijvers, and Gregory Neven (2018). “Compact Multi-Signatures for Smaller Blockchains”. In: *Advances in Cryptology – ASIACRYPT 2018, Part II*. Vol. 11273. Lecture Notes in Computer Science. Springer, pp. 435–464. DOI: [10.1007/978-3-030-03329-3_15](https://doi.org/10.1007/978-3-030-03329-3_15).
- Boneh, Dan, Craig Gentry, Ben Lynn, and Hovav Shacham (2003). “Aggregate and Verifiably Encrypted Signatures from Bilinear Maps”. In: *Advances in Cryptology – EUROCRYPT 2003*. Vol. 2656. Lecture Notes in Computer Science. Springer, pp. 416–432. DOI: [10.1007/3-540-39200-9_26](https://doi.org/10.1007/3-540-39200-9_26).
- Boneh, Dan, Ben Lynn, and Hovav Shacham (2001). “Short Signatures from the Weil Pairing”. In: *Advances in Cryptology – ASIACRYPT 2001*. Vol. 2248. Lecture Notes in Computer Science. Springer, pp. 514–532. DOI: [10.1007/3-540-45682-1_30](https://doi.org/10.1007/3-540-45682-1_30).
- Bühler, Christoph, Matteo Biagiola, Luca Di Grazia, and Guido Salvaneschi (June 2026). “Agent-Bound: Securing Execution Boundaries of AI Agents”. In: *Proceedings of the ACM on Software Engineering* 3.FSE, pp. 2141–2164. DOI: [10.1145/3808103](https://doi.org/10.1145/3808103).
- Ding, Zhiyang, Yang Luo, Guangpu Chen, Qingni Shen, and Zhonghai Wu (Sept. 2026). “ACLE-MCP: Attested Capability Leases for Execution-Time Trust in Remote LLM Tool Use”. In: *arXiv preprint arXiv:2609.02690*. URL: <https://arxiv.org/abs/2609.02690>.
- Eclipse Biscuit Project (2026). *Biscuit Authorization Token Specification*. Specification 3.x, Datalog version 3.3. Originally developed at Clever Cloud; now an Eclipse Foundation project. URL: <https://github.com/eclipse-biscuit/biscuit>.
- Ellison, Carl M., Bill Frantz, Butler Lampson, Ronald L. Rivest, Brian M. Thomas, and Tatu Ylonen (Sept. 1999). *SPKI Certificate Theory*. RFC 2693. DOI: [10.17487/RFC2693](https://doi.org/10.17487/RFC2693).
- Faz-Hernández, Armando, Sam Scott, Nick Sullivan, Riad S. Wahby, and Christopher A. Wood (Aug. 2023). *Hashing to Elliptic Curves*. RFC 9380. DOI: [10.17487/RFC9380](https://doi.org/10.17487/RFC9380).
- Hardt, Dick, Aaron Parecki, and Torsten Lodderstedt (Sept. 2026). *The OAuth 2.1 Authorization Framework*. Internet-Draft draft-ietf-oauth-v2-1-16. Work in progress. URL: <https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1-16/>.
- Laurie, Ben, Eran Messeri, and Rob Stradling (Dec. 2021). *Certificate Transparency Version 2.0*. RFC 9162. DOI: [10.17487/RFC9162](https://doi.org/10.17487/RFC9162).
- Lemmer-Webber, Christine, Dmitri Zagidulin, Manu Sporny, and Benjamin Goering (2026). *Authorization Capabilities*. W3C Credentials Community Group Draft Report, v0.4.0-rc.6. Commonly referred to as ZCAP-LD. URL: <https://w3c-ccg.github.io/zcap-spec/>.
- Llambí-Morillas, Mar and Daniel Fernández-Fernández (Sept. 2026). “Cryptographically Verifiable Authorization for Autonomous AI Agents: A Falsifiable Hypothesis and Proof of Concept”. In: *Frontiers in Computer Science* 8, 1966725. DOI: [10.3389/fcomp.2026.1966725](https://doi.org/10.3389/fcomp.2026.1966725).
- Lysyanskaya, Anna, Silvio Micali, Leonid Reyzin, and Hovav Shacham (2004). “Sequential Aggregate Signatures from Trapdoor Permutations”. In: *Advances in Cryptology – EUROCRYPT 2004*. Vol. 3027. Lecture Notes in Computer Science. Springer, pp. 74–90. DOI: [10.1007/978-3-540-24676-3_5](https://doi.org/10.1007/978-3-540-24676-3_5).
- Maloyan, Narek and Dmitry Namiot (Jan. 2026). “Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents”. In: *arXiv preprint arXiv:2601.17549*. URL: <https://arxiv.org/abs/2601.17549>.
- Metere, Alfredo (May 2026). “Attested Tool-Server Admission: A Security Extension to the Model Context Protocol”. In: *arXiv preprint arXiv:2605.24248*. URL: <https://arxiv.org/abs/2605.24248>.
- Miller, Mark Samuel (May 2006). “Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control”. PhD thesis. Baltimore, Maryland: Johns Hopkins University.
- Model Context Protocol Project (July 2026). *Model Context Protocol Specification*. Revision 2026-07-28. Introduced by Anthropic in November 2024; stewarded since December 2025

- by the Agentic AI Foundation, a directed fund of the Linux Foundation. URL: <https://modelcontextprotocol.io/specification/2026-07-28>.
- Prakash, Sunil (Mar. 2026). “AIP: Agent Identity Protocol for Verifiable Delegation Across MCP and A2A”. In: *arXiv preprint arXiv:2603.24775*. URL: <https://arxiv.org/abs/2603.24775>.
- Ristenpart, Thomas and Scott Yilek (2007). “The Power of Proofs-of-Possession: Securing Multiparty Signatures against Rogue-Key Attacks”. In: *Advances in Cryptology – EURO-CRYPT 2007*. Vol. 4515. Lecture Notes in Computer Science. Springer, pp. 228–245. DOI: [10.1007/978-3-540-72540-4_13](https://doi.org/10.1007/978-3-540-72540-4_13).
- Sakemi, Yumi, Satoru Kanno, and Riad S. Wahby (Sept. 2026). *Pairing-Friendly Curves*. Internet-Draft draft-irtf-cfrg-pairing-friendly-curves-14. Work in progress. URL: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-pairing-friendly-curves/14/>.
- Schwenkschuster, Arndt, Pieter Kasselmann, Kelley Burgin, Michael J. Jenkins, Brian Campbell, and Aaron Parecki (July 2026). *OAuth Identity and Authorization Chaining Across Domains*. Internet-Draft draft-ietf-oauth-identity-chaining-17. Work in progress. URL: <https://datatracker.ietf.org/doc/draft-ietf-oauth-identity-chaining/17/>.
- SPIFFE Project (2026). *The SPIFFE Specifications*. Specification set, stability designation “Stable”. The SPIFFE standards carry stability designations rather than release numbers. URL: <https://github.com/spiffe/spiffe/tree/main/standards>.
- Tuecke, Steven, Von Welch, Doug Engert, Laura Pearlman, and Mary Thompson (June 2004). *Internet X.509 Public Key Infrastructure (PKI) Proxy Certificate Profile*. RFC 3820. DOI: [10.17487/RFC3820](https://doi.org/10.17487/RFC3820).
- UCAN Working Group (2026). *User Controlled Authorization Networks (UCAN) Specification*. Version 1.0.0. URL: <https://github.com/ucan-wg/spec>.
- Zhou, Huijun, Xiaohan Zhang, Haozhe Zhang, Haoyang Zhang, Mi Zhang, and Min Yang (May 2026). “A First Measurement Study on Authentication Security in Real-World Remote MCP Servers”. In: *arXiv preprint arXiv:2605.22333*. URL: <https://arxiv.org/abs/2605.22333>.
- Zhou, Ziling (Mar. 2026). “Governing Dynamic Capabilities: Cryptographic Binding and Reproducibility Verification for AI Agent Tool Use”. In: *arXiv preprint arXiv:2603.14332*. URL: <https://arxiv.org/abs/2603.14332>.